
Technique de la robotique

cours d'introduction I - version 2015.09

Les règles de sécurité

Le robot est considéré dans le milieu industriel comme étant une machine dangereuse (classe 4/4). Cela signifie qu'il est capable de générer des lésions graves et irréversibles sur l'être humain. Ce risque doit être pris au sérieux même si les commandes actuelles sont extrêmement fiables. Afin d'éviter toute malheureuse rencontre avec le bras du robot, il est impératif de respecter rigoureusement les 10 règles de sécurité suivantes:



1. Un robot est une machine dangereuse.
2. Ne jamais laisser un robot seul sous puissance.
3. Ne jamais s'asseoir, ni lire, ni écrire sur les tables des robots.
4. Tester les boutons d'**ARRÊT D'URGENCE** lors de la première mise en puissance.
5. Imposer dès le départ une vitesse très réduite.
6. Avoir la main sur le bouton d'**ARRÊT D'URGENCE** lors de chaque déplacement du robot.
7. Ne jamais se trouver dans la zone de volume de préhension du robot lorsque ce dernier exécute un programme.
8. Réfléchir aux conséquences des actions déclenchées avec le clavier ou la souris, et veiller aux autres personnes présentes près de l'application.
9. Ne pas exécuter des programmes autres que les siens sans autorisation préalable.
10. Avertir immédiatement les responsables lors d'incidents, même mineurs.

De plus, il faut aussi respecter les deux règles suivantes :

- Avant toute intervention sur le bras, vous devez également couper l'alimentation des moteurs.
- Avant toute intervention dans l'armoire de commande, vous devez couper l'alimentation générale de la machine.

Ne jamais travailler seul avec une installation robotique. Il faut au minimum 2 personnes présentes dans le laboratoire pour pouvoir utiliser un robot.

1	Introduction.....	1
1.1	Marché de la robotique industrielle.....	2
2	Les robots dans le monde industriel	4
2.1	Généralités	4
2.2	Domaines d'applications	8
2.3	Assemblage robotisé	10
2.3.1	L'assemblage manuel	11
2.3.2	Assemblage automatique.....	12
2.3.3	Assemblage robotique.....	13
3	Les robots industriels	14
3.1	Les caractéristiques des robots industriels	14
3.1.1	Le nombre de degrés de liberté	15
3.1.2	La répétabilité.....	15
3.1.3	L'enveloppe de travail	16
3.1.4	La charge transportable admise	17
3.1.5	La vitesse maximum de déplacement	18
3.1.6	L'environnement de travail	19
3.1.7	La compliance (ou rigidité)	19
3.1.8	Le système de commande.....	20
4	Le bras (partie mécanique).....	21
4.1	Les types d'architectures.....	21
4.2	L'architecture Série	22
4.2.1	Les robots autoportiques ou cartésiens	22
4.2.2	Les robots SCARA.....	23
4.2.3	Les robots anthropomorphes.....	25
4.3	L'architecture hybride	27
4.3.1	Le robot delta ABB IRB 340	27
4.3.2	Le robot Fanuc M-1iA	29
4.3.3	Le robot Mitsubishi de type RP-5AH	30
4.3.4	L'hexapode	31
4.4	Les robots coopératifs	32
5	La commande (partie électronique).....	33
6	Systèmes de coordonnées.....	34
6.1	Calculations <i>directe</i> et <i>inverse</i>	34
6.2	Le système de coordonnées articulaire.....	35
6.3	Le système de coordonnées cartésien	36
6.3.1	Système cartésien - les matrices de positions.....	37

6.3.2	Exemple de rotation: les angles de Cardan - Yaw-Pitch-Roll.....	39
6.4	La méthode Denavit-Hartenberg.....	42
6.5	La configuration du bras.....	43
6.6	Singularités	45
6.7	Apprentissage d'une position orientée avec le robot	46
6.7.1	frame plan	46
6.7.2	frame cylindrique	46
7	Mouvements programmés avec Synapxis.....	47
7.1	Mouvement articulaire.....	47
7.2	Mouvement cartésien	47
7.2.1	Tool.....	47
7.2.2	Mouvement cartésien - interpolation joint.....	47
7.2.3	Mouvement linéaire	48
7.2.4	Mouvement circulaire	48
7.2.5	Frames	48
7.3	Vitesses et accélérations	48
7.3.1	Vitesse moniteur	48
7.3.2	Vitesse programme	48
7.3.3	Vitesse linéaire	49
7.3.4	Accélération et décélération	49
7.4	Enchaînement de mouvements	49
7.4.1	Synchronisation	49
7.4.2	Arrêt au point	50
7.4.3	Passage au point.....	51
8	Programmation	52
8.1	Système informatique : les rudiments de son fonctionnement	52
8.1.1	Cœur.....	52
8.1.2	Processeur	52
8.1.3	Multitâches.....	52
8.1.4	Processus.....	53
8.1.5	Thread.....	53
8.1.6	Mémoire.....	53
8.2	Rappels des notions fondamentales de programmation	56
8.2.1	Type de programmation.....	56
8.2.2	Programme.....	56
8.2.3	Procédures.....	57
8.2.4	Paramètres	57

8.2.5	Fonction et type de retour	58
8.2.6	Valeur et référence.....	58
8.2.7	Types de variables	59
8.2.8	Variables locales	60
8.2.9	Variables globales.....	61
8.2.10	Syntaxe	62
8.3	Programmation à l'aide de Synapxis	63
8.3.1	Dossier et Fichiers	63
8.3.2	Procédure principale	63
8.3.3	Variables globales : temporaire ou persistante ?.....	65
8.4	Charte syntaxique.....	65
9	Réseaux, bus de terrain, IOs.....	66
9.1	TCP/IP	66
9.1.1	Internet.....	66
9.1.2	Réseau local, Ethernet, Wi-Fi	66
9.1.3	Serveur	67
9.1.4	Client.....	68
9.1.5	Adresse physique (MAC)	68
9.1.6	Adresse IP	68
9.1.7	Port	69
9.1.8	Switch Ethernet	70
9.1.9	Hub Ethernet	70
9.1.10	Câble croisé	70
9.1.11	DNS, DynDNS	70
9.1.12	Visualiser et manipuler les paramètres réseau sous Windows.....	71
10	Des connaissances utiles à l'ingénieur	74
10.1	Loi des similitudes, ou "effet d'échelle"	74
	Sources.....	76

1 Introduction

Avant de se lancer dans des définitions trop techniques, il est important de situer le contexte. Le monde de la robotique a plusieurs aspects et cela commence avec les robots "jouets" en passant par les robots "sonde" (Curiosity, sur mars depuis août 2012) sans oublier les robots ménagers ainsi que les robots industriels (RI). Par conséquent, il apparaît que le mot "robot" possède plusieurs définitions. Celle donnée par le Petit Larousse est la suivante:

Robot (du tchèque robota, travail forcé, corvée)

- *Dans les œuvres de science-fiction, machine à l'aspect humain, capable de se mouvoir, d'exécuter des opérations, de parler.*
- *Appareil automatique capable de manipuler des objets ou d'exécuter des opérations selon un programme fixe ou modifiable, voire par apprentissage; son déplacement résulte d'une combinaison de mouvements autour de six axes différents.*
- *Bloc-moteur électrique combinable avec divers accessoires, destiné à différentes opérations culinaires.*
- *Fig. Personne qui agit de façon automatique.*

Dans ce cours, c'est la 2^{ème} définition qui est étudiée, c'est-à-dire celle qui concerne les robots industriels (RI). Il s'avèrera que la définition donnée par le Petit Larousse ne correspond pas tout à fait à la réalité de la robotique telle que conçue ici.

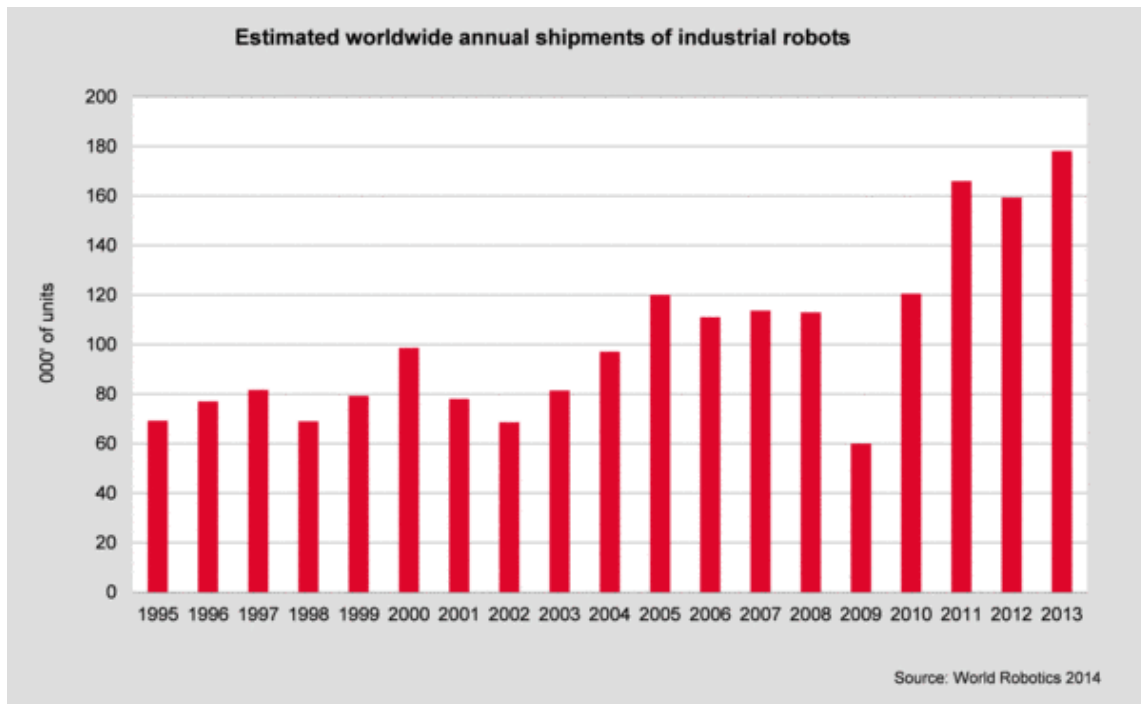
L'utilisation des RI dans notre sphère économique touche plusieurs domaines d'activités :

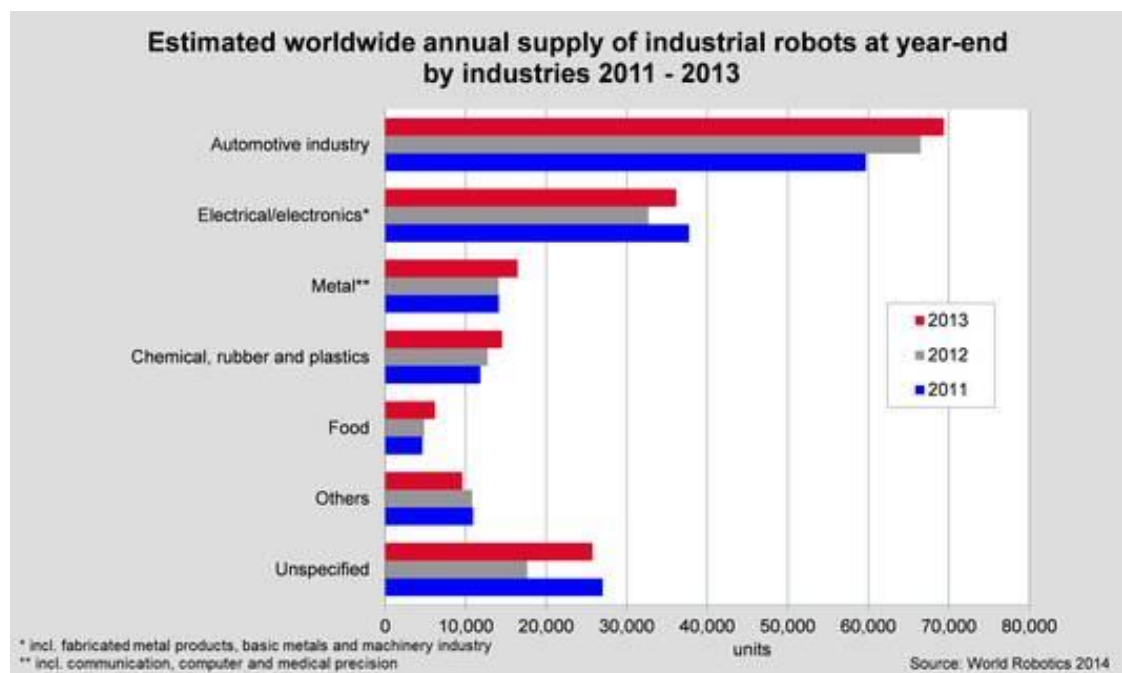
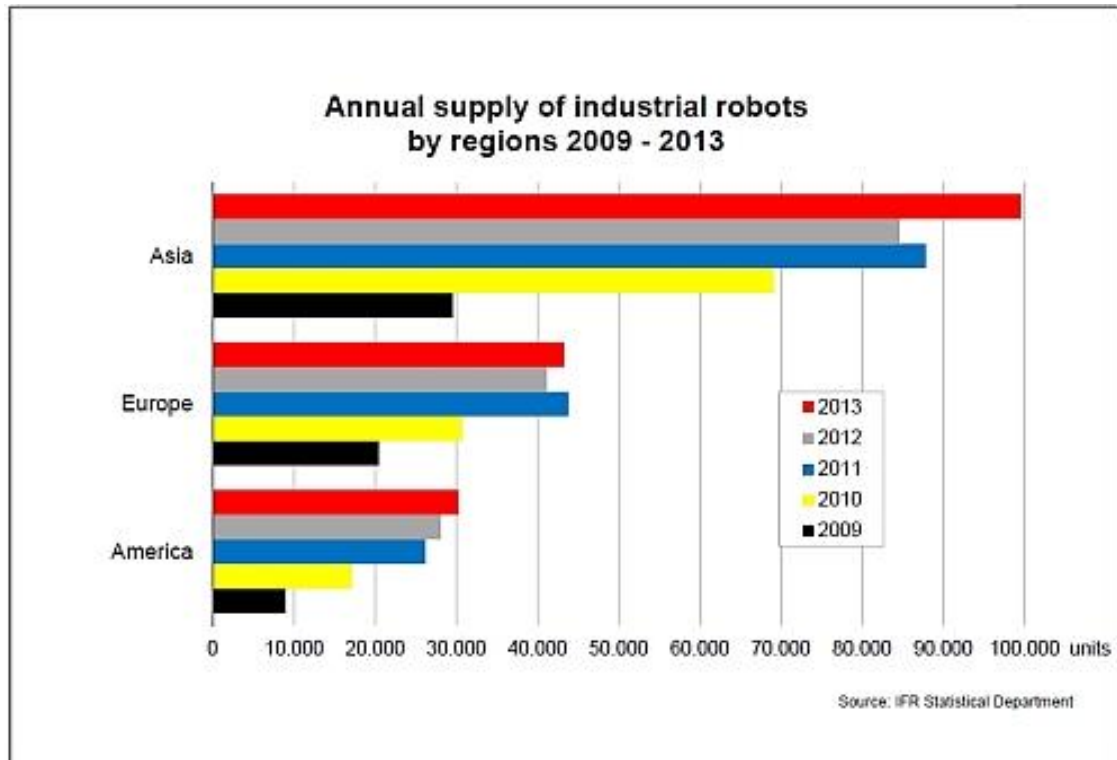
- Dans le domaine de la production de masse (assemblage, chargement de machine, manipulation de pièces, etc...).
- Manipulation en milieu hostile et dangereux pour l'homme (manipulation en milieu radioactif, en milieu explosif, etc...).
- Utilisation dans le but de générer et maîtriser de mouvements complexes dans l'espace (suivi de trajectoire, opération chirurgicale etc...).

Cette liste n'est pas exhaustive et l'évolution des machines va permettre d'explorer de nouveaux domaines d'applications.

1.1 Marché de la robotique industrielle

A titre indicatif, voici un aperçu de l'évolution des ventes de robots industriels dans le monde. La croissance de ce marché est stimulée par des entreprises telles que Foxconn, actives dans la production d'appareil électroniques (Dell, Apple, Samsung, HP, Sony, Lenovo, Nokia, etc.), qui commencent à automatiser des tâches de productions pourtant réalisées par de la main d'œuvre très bon marché (Chine, Algérie, etc).



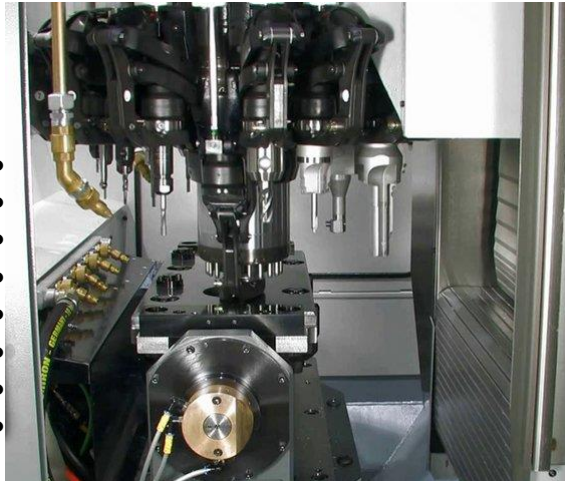


Sources :
<http://www.ifr.org/industrial-robots/statistics/>
<http://www.manmonthly.com.au/News/industrial-robot-sales-at-an-all-time-high>
<https://fr.wikipedia.org/wiki/Foxconn>

2 Les robots dans le monde industriel

2.1 Généralités

- Le robot se distingue des *Machines Outil* notamment grâce à son enveloppe de travail, qui est ouverte vers l'extérieur. Son enveloppe de travail lui permet par exemple de saisir une pièce au vol sur un convoyeur avant de réaliser le processus sur la pièce.



CNC - volume accessible fermé



Robot - volume accessible ouvert

- Les robots ont une rigidité moins grande que les *Machines Outils* de par leur structure et la nature de leur transmission/réducteur. La transmission de mouvement avec des courroies crantées, cascade d'engrenages ou cardans sont notamment des éléments qui permettent de diminuer les masses en mouvement (inertie) au détriment de la rigidité.
- L'ensemble des robots du marché couvre un rayon d'action (ou portée) allant de quelques dizaine de cm à plusieurs mètres (4m), avec une charge utile allant de moins d'un kg à plus d'une tonne (2012).



Mitsubishi RV1A, 1kg, 410mm



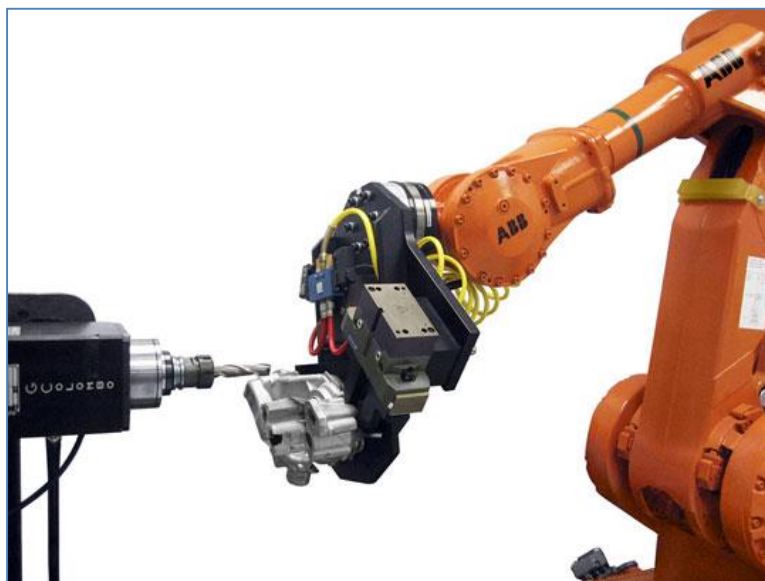
Kuka KR1000, 1000kg, 3600mm

- Dans la plus part des cas, un robot est monté au sol. Certains modèles de bras peuvent être montés au sol, au plafond ou au mur. Cette configuration doit parfois être renseignée au niveau du contrôleur pour une compensation logicielle de la gravité.
- Un robot peut exécuter des déplacements "points à points" ou suivre une trajectoire donnée, lui permettant de réaliser des applications industriels très diverses.
- Un robot peut travailler selon plusieurs configurations distinctes :
 - pièce fixe : le robot porte l'outil avec lequel il doit exécuter une opération (perçage, soudage, usinage)



Découpe en pièce fixe

- pièce portée : le robot porte la pièce sur laquelle il doit exécuter des opérations (polissage, meulage, découpe, etc). L'avantage de cette configuration est qu'il peut réaliser la partie handling (palettisation, suivi de convoyeur, etc).



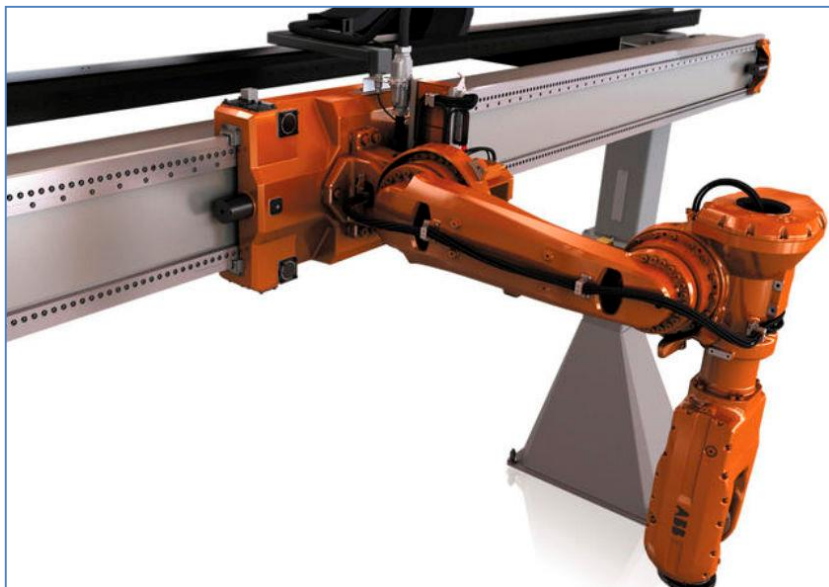
Usinage en pièce portée

- collaboration : un ou plusieurs robot portent une pièce pendant que d'autres effectuent des opérations sur celle-ci (soudage, etc).



Coopération: 2 robots portent la charge, 2 autres travaillent dessus

- Un robot peut être combiné avec un ou plusieurs *axes externes*, afin d'augmenter son accessibilité :
 - robot monté sur un axe linéaire, lui permettant de se déplacer sur une distance importante.



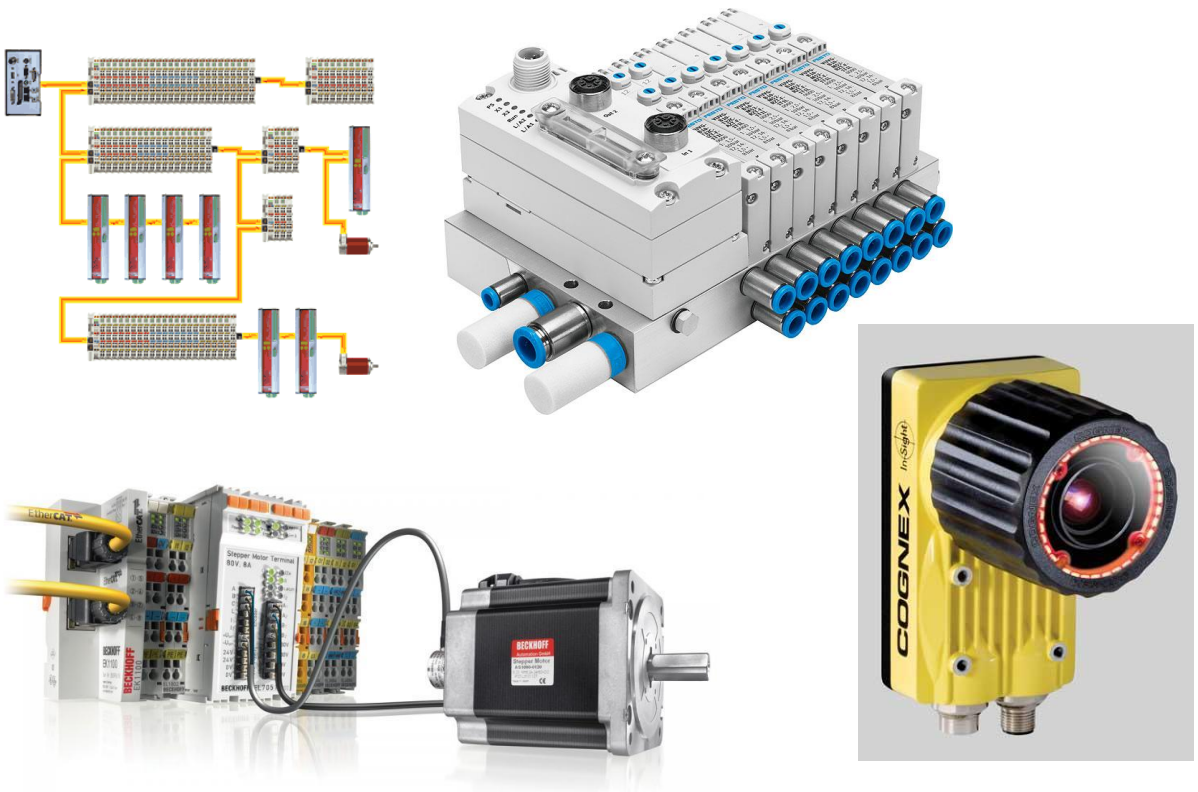
Robot monté sur un axe linéaire

- un axe rotatif fait tourner une pièce devant le robot pour que celui-ci puisse accéder à l'ensemble de ses faces.



La pièce est positionnée par un axe rotatif

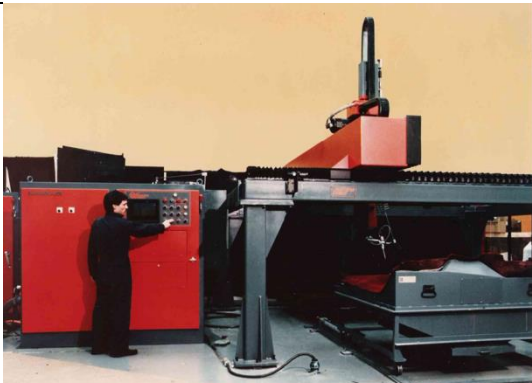
- Un robot peut interagir avec de nombreux éléments péri-robotiques, afin d'augmenter ses fonctionnalités :
 - les standard : relais, électrovannes, capteurs, etc.
 - bus IOs : DeviceNet, Profibus, Ethercat, Ethernet, Série, etc.
 - caméras
 - automate programmables (API)
 - autre robots, CNC
 - ...



2.2 Domaines d'applications

La robotique industrielle s'applique à de très nombreux et vastes domaines, dont voici quelques exemples :

handling	alimentation, bâtiment, fonderie,		
suivi de convoyeur (tracking)	alimentation		
assemblage	automobile, médicale		
usinage	prototypage, industrie du bois,		

traitement de surface	meulage, polissage, satinage		
opérations industrielles	pliage, découpage plasma, découpage jet d'eau, soudure		
alimentaire	boucherie, packaging		
recherche	médical		
milieux hostile	radioactivité, explosivité, chaleur, vide		

2.3 Assemblage robotisé

Aujourd'hui, la concurrence est rude et si une entreprise veut lancer un produit sur le marché, elle doit vérifier 4 paramètres fondamentaux.

- Le rapport qualité prix doit être concurrentiel.
- Le produit doit être fonctionnel.
- Le produit doit répondre à un réel besoin du client.
- Le design doit être « tendance »

Si un de ces paramètres n'est pas respecté, l'entreprise va au-delà d'un échec qui peut remettre en cause son existence. De plus, si le premier paramètre est considéré, il est important que le coût soit le plus faible possible. Or lorsque qu'il apparaît que le 80% du prix d'un produit est imputé à son assemblage ou plutôt à sa mise en production, l'endroit où les efforts doivent être mis devient évident.

Aujourd'hui, une société qui désire lancer un produit de grande consommation sur le marché doit être particulièrement attentive à son mode d'assemblage. Comme le coût de la main d'œuvre est très onéreux dans les pays industrialisés, l'entreprise en question va se retrouver confrontée à trois solutions pour assembler son produit.

- Assemblage manuel (délocalisation de la production)
- Assemblage automatique (dédié à un produit)
- Assemblage robotique (flexible)

Dans le cas extrême, l'entreprise peut se retrouver dans le cas où elle doit purement et simplement refuser de mettre son produit en fabrication et cela uniquement pour des raisons de coût de production.

2.3.1 L'assemblage manuel

L'assemblage manuel est encore très utilisé aujourd'hui, en effet, le faible coût de la main d'œuvre dans les pays en voie d'industrialisation encourage fortement la délocalisation de la production. Cette solution est la plus simple au premier abord. Cependant, il ne faut pas négliger le choc des cultures. La formation des ouvriers(ères) n'est pas toujours aisée dans ces pays et des problèmes de qualité peuvent survenir et mettre en péril le renom d'un produit ou d'une marque.

Assemblage manuel	
Avantages	Inconvénients
Flexibilité (Rapidité avec laquelle un nouveau produit peut être mis en production)	Cadence (Cadence de production relativement faible et qui peut-être difficilement augmentée)
Temps de lancement T_L (Temps relativement court entre la mise au point du prototype et celui de la mise sur le marché.)	Coût important de la main d'oeuvre (Le coût est très important dans les pays industrialisés et cela nécessite dans la majorité des cas, une délocalisation complète de la production à moins d'un besoin d'une qualification particulière.)
Coût d'investissement à court terme (Si le produit n'a pas le succès escompté sur le marché, l'entreprise peut arrêter la production dans un temps très court et utiliser les ouvriers pour effectuer une autre tâche.)	Qualité non constante (L'être humain en tant que tel est incapable de répéter plusieurs opérations de manière à ce qu'elles soient parfaitement semblables.)



2.3.2 Assemblage automatique

L'assemblage automatique est souvent appelé assemblage dédié. En effet, la machine est construite autour d'un unique produit. Par conséquent, elle est dédiée à un produit et une modification de ce dernier ou une mise en série d'un nouveau produit rend la machine parfaitement inutilisable. Pour se lancer dans l'achat d'une telle machine, il faut s'assurer d'un paramètre fondamental.

Pièces à produire par année > 1×10^6 pièces

Ces machines de production fonctionnent pour la plupart à l'aide de cames, par conséquent, les cadences sont très importantes. Il s'agit de plusieurs pièces par seconde. Le canton de Neuchâtel abrite deux entreprises phares dans la construction de ce genre de machines. Il s'agit des sociétés Ismeca SA et Mikron SA.

Assemblage automatique	
Avantages	Inconvénients
Cadence (Cadence de production extrêmement élevée et facilement réglable.)	Dédié à un produit (Il est quasi impossible d'adapter la machine suite à une modification du produit.)
Qualité constante (Une machine ne possède pas d'état d'âme, par conséquent, elle travaille toujours de façon parfaitement identique. La qualité est remarquablement constante. Une qualité non maîtrisée coûte très cher aux entreprises.)	Coût d'investissement à court terme (Lorsqu'une entreprise décide d'acquérir une chaîne automatique, elle doit prendre des risques financiers. Si le produit ne se vend pas, la société devra avoir les reins solides afin de pouvoir absorber la perte financière.)
Travail 24/24 (Le travail de nuit, le dimanche et les jours fériés est très réglementé et plus cher pour l'entreprise. Réglementation qui ne s'applique pas aux machines)	Temps de lancement T_L (Temps Très important entre la décision de fabriquer la machine et la mise du produit sur le marché.)



2.3.3 Assemblage robotique

L'assemblage robotique est en pleine expansion. En effet, l'évolution des techniques informatiques a permis aux machines d'évoluer tant au niveau des performances physiques qu'au niveau de la convivialité. Si des erreurs importantes ont été faites dans le passé (robotisation à outrance sans réflexion sur les possibilités réelles des machines), aujourd'hui la robotique est repartie sur des bases saines et les perspectives pour le futur sont des plus optimistes. L'avantage principal des robots est sans aucun doute leur grande flexibilité **par rapport à l'assemblage automatique**. En effet, ces machines programmables peuvent s'adapter relativement facilement à une modification ou un changement d'application si cela a été prévu à la conception de la machine.

Il est toutefois important de signaler que le robot n'est pas une machine universelle. Chaque robot possède plusieurs caractéristiques et il faut choisir le bon robot pour l'application considérée.

Les robots ne sont pas uniquement utilisés dans le monde de l'assemblage, mais également dans d'autres domaines d'applications comme par exemple: le suivi de trajectoire (collage, soudage, découpe jet d'eau, etc...), les manipulations en milieu hostile (manipulation des barres d'uranium dans les centrales nucléaires, etc...).

Assemblage robotique	
Avantages	Inconvénients
« Flexibilité » (Rapidité avec laquelle une nouvelle variante du produit peut être mise en production, si prévue initialement dans le cahier des charges de la machine.)	Cadence (Cadence relativement faible par rapport à la machine automatique. Le temps de cycle technique t_{ct} est à peu près équivalent à celui de l'être humain.)
Qualité constante (Une machine ne possède pas d'état d'âme, par conséquent, elle travaille toujours de façon parfaitement identique. La qualité est constante. Une qualité non maîtrisée coûte très cher aux entreprises.)	Coût d'investissement à court terme (Lorsqu'une entreprise décide d'acquérir une cellule robotique, elle doit prendre des risques financiers. Si le produit ne se vend pas, la société devra avoir les reins solides afin de pouvoir absorber la perte financière.)
Travail 24/24 (Le travail de nuit, le dimanche et les jours fériés est très réglementé et plus cher pour l'entreprise. Réglementation qui ne s'applique pas aux machines)	Temps de lancement T_L (Temps relativement important entre la décision de fabriquer la cellule ou la chaîne robotisée et la mise du produit sur le marché.)

Ces tableaux démontrent les possibilités de chacune des méthodes décrites ci-dessus. L'entreprise devra effectuer un choix judicieux en tenant compte de tous les paramètres possibles afin d'obtenir la solution la mieux adaptée au produit à industrialiser.



3 Les robots industriels

Actuellement, il existe un grand nombre de robots industriels de types et de marques différentes sur le marché. Comme dans les autres branches industrielles, les fusions ou les regroupements sont de mise, et par conséquent, le monde de la robotique se voit confronter à une diminution du nombre de fabricants. Il n'est pas facile, voire impossible pour un constructeur de maîtriser la conception de tous les types de bras et de commande, c'est pour cette raison que chaque marque a des spécificités propres, connues dans le monde de la robotique.

Les robots industriels sont composés de deux éléments essentiels :

- le bras (partie mécanique de la machine),
- la commande (partie électronique qui gère le déplacement du bras).

3.1 Les caractéristiques des robots industriels

Chaque application demande certaines exigences de la part des robots. Par conséquent, le choix du robot revêt de la première importance pour la réussite de l'application. De plus, il est important de signaler que :

Il n'y a pas de bons ou de mauvais robots mais il y a de bonnes ou de mauvaises applications avec ceux-ci.

MEI

Le choix du robot va être déterminé par les 8 caractéristiques suivantes:

- Le nombre de degrés de liberté
- La répétabilité
- L'enveloppe de travail
- La charge transportable admise
- « Vitesse maximum par axe en °/s ou vitesse en mouvement cartésien en m/s »
- L'environnement de travail
- La compliance (ou rigidité)
- Le système de commande (CPU)

3.1.1 Le nombre de degrés de liberté

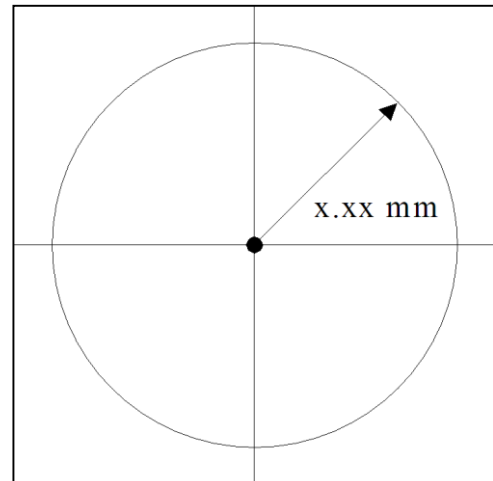
Le “ nombre de degré de liberté d’un robot” correspond au nombre d’axes que le bras possède. Chaque axe est muni d’un moteur, d’un codeur et d’un réducteur. La majorité des robots existants sur le marché possède entre 3 et 6 degrés de liberté. Cependant, il est possible de trouver des robots possédant un nombre de degré de liberté supérieur à 6 ou inférieur à 3. Le mouvement généré par un vérin pneumatique (par exemple la pince du robot) ne compte pas pour un degré de liberté supplémentaire.

3.1.2 La répétabilité

Le mot répétabilité ne se trouve pas dans le dictionnaire. Cependant, la littérature parle toujours de la répétitivité d’un bras. C’est une notion purement statistique qui est influencée par 3 incertitudes : Les erreurs mécaniques, les erreurs dues aux codeurs, à l’électronique et les erreurs des algorithmes de calcul de trajectoires.

La valeur donnée représente le rayon de la cible dans laquelle le robot s’arrête après plus de 1000 déplacements appris à différents endroits représentatifs de l’enveloppe de travail. Pour l’intégrateur, cette valeur donne la tolérance dans laquelle le constructeur garantit que son robot est capable de revenir se positionner à un point créé, soit par apprentissage, soit par calcul (*voir figure ci-contre*).

Si le constructeur donne une répétabilité de ± 0.05 mm, il est possible d’affirmer que pour un point donné, l’extrémité du bras se positionnera toujours à l’intérieur d’une cible de 0.05 mm de rayon.

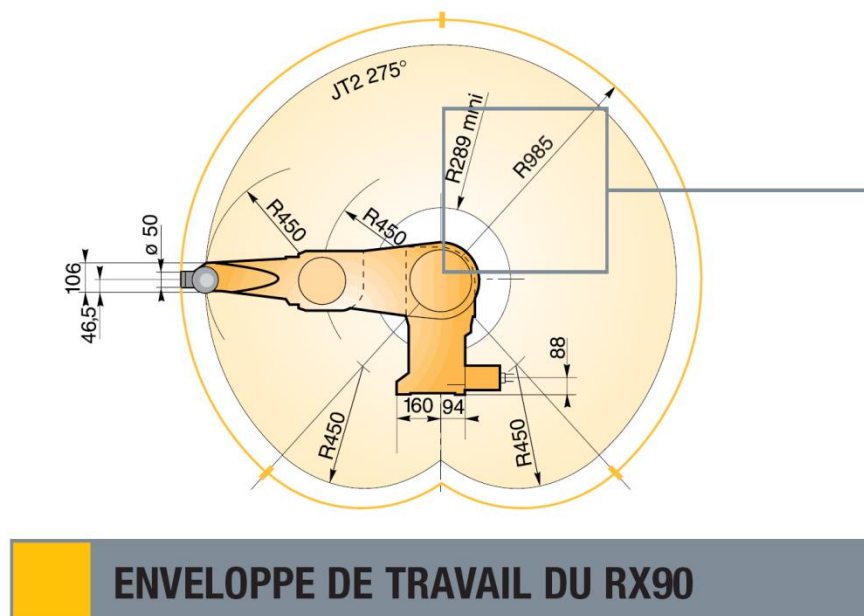


3.1.3 L'enveloppe de travail

L'enveloppe de travail d'un robot est définie par tous les points de l'espace qui peuvent être atteints par l'extrémité du robot (interface mécanique sans prendre en compte le préhenseur). L'enveloppe de travail ou du volume de préhension d'un robot ne tient pas compte des singularités¹! Chaque type de robot a son propre volume de préhension et il est défini par sa conception ainsi que par son nombre d'axes. Un robot anthropomorphe comportant 6 axes possède comme enveloppe de travail, une sphère creuse (voir figure ci-dessous). D'où le terme de robot sphérique.

¹ Une singularité est un point particulier à la limite entre 2 configurations physiques du bras du robot (flip-noFlip, above-below) dont la caractéristique est que deux axes de ce dernier se trouvent alignés (les deux axes de rotation sur la même droite). C'est une caractéristique propre à tous les robots 6 axes.

Pour passer un point singulier, il faut souvent effectuer un mouvement ample et rapide des axes entraînant souvent un blocage de sécurité. Cependant certains contrôleurs sont plus aptes que d'autres à gérer les singularités, notamment par une gestion des vitesses et des accélérations appropriée.



L'image ci-dessus montre clairement les limites de l'enveloppe. Cependant, en règle générale, il faut réaliser l'implantation de façon à travailler sur le plus grand rayon du robot, donc à l'altitude $Z = 0$ (centre de l'axe 2 selon la figure). De plus, il ne faut pas faire une implantation avec le robot en limite d'enveloppe. Comme l'être humain, le robot a du mal à travailler le bras tendu. L'idéal est de travailler aux 2/3 de l'enveloppe.

3.1.4 La charge transportable admise

La charge transportable admise comprend toute la mécanique qui va être ajoutée à l'extrémité du bras. Elle est définie par la loi suivante:

$$\text{Charge} = \text{masse du préhenseur} + \text{masse de la pièce à transporter} + \text{contraintes externes (câbles, tuyaux, etc...)}$$

La littérature donne une indication concernant la charge admise par le bras. La charge peut varier de quelques grammes à plusieurs centaines de kilos. Plus la charge admise est élevée plus le robot est grand ce qui a pour conséquence, en général et pour une même gamme de robot, une moins bonne répétabilité.

Avant d'intégrer un robot dans une application, il faut prendre garde à bien évaluer, quantifier, mesurer tous ces paramètres. Si les valeurs maximales admises ne sont pas respectées, le robot ne pourra pas être utilisé à 100% de ses possibilités. Par conséquent, les performances (vitesse, comportement) de l'application peuvent être limitées si l'un des paramètres suivant n'est pas respecté: la masse mais également, le couple statique et l'inertie.

Caractéristiques de la charge :

Position au centre de gravité de la charge (M) : $z = 150 \text{ mm}$ par rapport à l'axe 5 et $x = 75 \text{ mm}$ par rapport à l'axe 6 (figure 2.1.5).

Charge transportable	Bras standard	Bras long
A vitesse nominale*	6 kg	3,5 kg
A vitesse réduite*	9 kg	6 kg
Dans certaines configurations (nous consulter)	12 kg	9 kg

* dans toutes les configurations et en tenant compte des inerties maximales. Voir tableau ci-dessous.

	Inerties nominales (kg.m ²)		Inerties maximales (kg.m ²)**	
	Bras standard	Bras long	Bras standard	Bras long
Par rapport à l'axe 5	0,135	0,080	0,675	0,400
Par rapport à l'axe 6	0,034	0,020	0,170	0,100

** dans les conditions de vitesse et accélération réduites : SP 60 maxi et ACC (8) 50,50.

1.4.1. COUPLES LIMITES

	Axe référence		
	Axe 5 (Z ₆)		Axe 6 (Z ₇)
Couple statique (Nm)	24 ⁽¹⁾	14 ⁽²⁾	10
Couple pointe (N.m)	100 ⁽¹⁾	57 ⁽²⁾	43

⁽¹⁾ si couple axe 6 = 0
⁽²⁾ si couple maximum sur axe 6

Exemple pour un robot Stäubli RX90B

3.1.5 La vitesse maximum de déplacement

Cette caractéristique est extrêmement importante car il ne faut pas oublier que le robot est une machine de production et plus la machine sera rapide plus vite elle sera rentabilisée.

Il existe deux types de vitesse, la vitesse maximum dans le repère articulaire donnée pour chaque axe en [°/s] et la vitesse maximum dans le repère cartésien donnée en [m/s] ou [mm/s]

Il est à noter qu'il n'est pas facile d'augmenter les vitesses sans pénaliser la répétabilité et/ou la rigidité des robots. De fait, plus le nombre de degré de liberté est faible, plus la vitesse peut-être élevée.

1.4.2. AMPLITUDE, VITESSE ET RESOLUTION

Axe	1	2	3	4 ⁽¹⁾	5	6
Amplitude (°)	320	275	285	540	225	540 ⁽²⁾
Répartition d'amplitude (°)	A +/-160	B +/-137.5	C +/-142.5	D +/-270	E +120 -105	F +/-270
Vitesse nominale (°/s)	236	200	286	401	320	580
Vitesse maximale (°/s)	356	356	296	409	800	1125 ⁽³⁾
Résolution angulaire (°.10 ⁻³)	0.87	0.87	0.72	1	1.95	2.75

⁽¹⁾pour les bras 5 axes, l'axe 4 est fixe. L'axe 5 correspond à l'axe 4 et l'axe 6 à l'axe 5 du logiciel.

⁽²⁾version multitours disponible en option.

⁽³⁾sans interaction de l'axe 5.

Vitesse réduite au pendant d'apprentissage : mode cartésien 250 mm/s

mode articulaire : 10% des vitesses nominales

Vitesse cartésienne maxi : 2 m/s



Dans certaines configurations du bras, les vitesses articulaires maxi ne peuvent être atteintes que si les charges et inerties sont réduites.

Exemple pour un robot Stäubli RX90B

3.1.6 L'environnement de travail

Le choix du robot doit également tenir compte du milieu dans lequel il va être intégré :

- Ambiance salle blanche (robot parfaitement étanche aux poussières)
- Ambiance explosive (moteurs spéciaux qui ne produisent pas d'étincelles)
- Ambiance agro-alimentaire (le bras doit être parfaitement étanche à toute fuite d'huile et pouvoir être lavé à grande eau)
- Ambiance avec pollutions extérieures (peintures, eau, etc...)
- Rayonnements (utilisation dans le nucléaire, rayonnement magnétique, rayonnement X, etc...)
- Températures basses ou élevées



3.1.7 La compliance (ou rigidité)

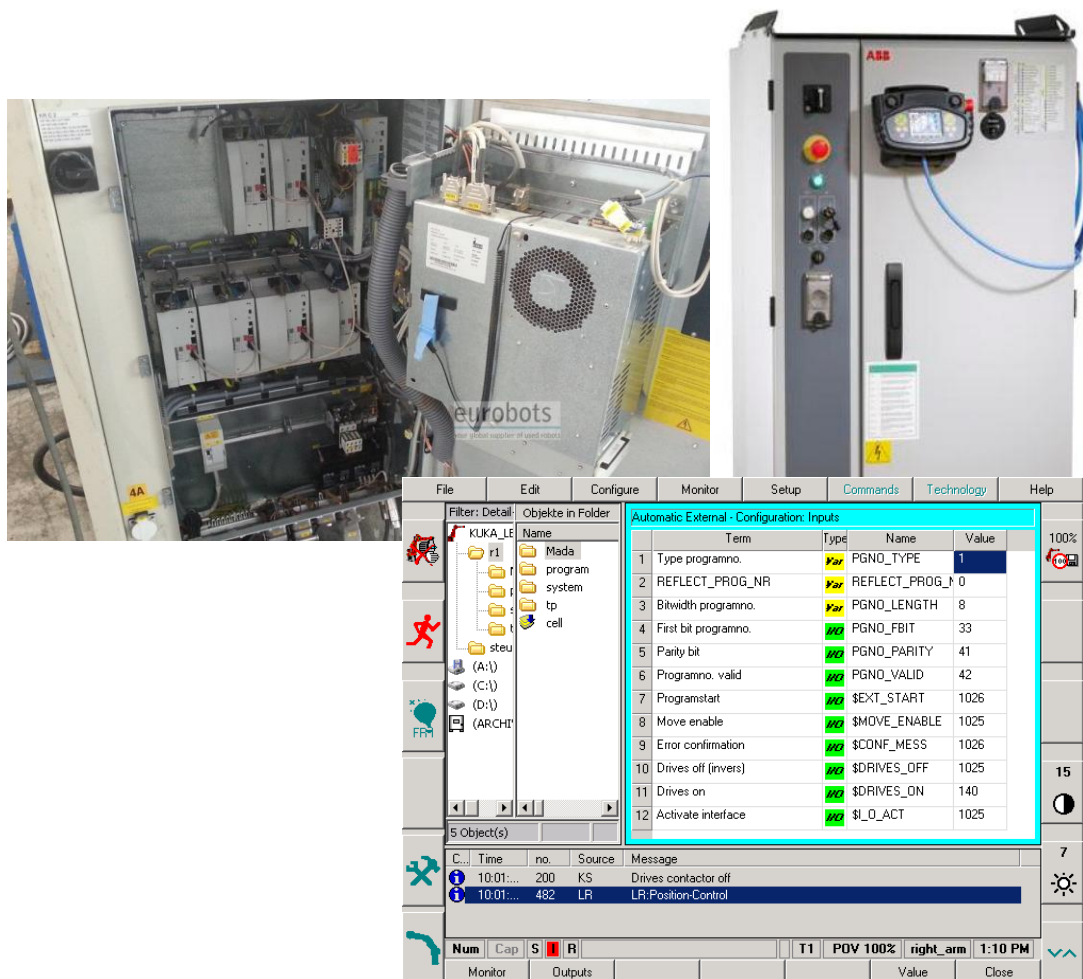
La compliance est l'inverse de la rigidité. Afin d'avoir la meilleure répétabilité et précision possibles, les constructeurs de robots ont tendance à faire des bras le plus rigide possible. Cependant pour certaines applications, la compliance peut aussi être un avantage. C'est pourquoi, la tendance actuelle des constructeurs de robots (Stäubli et Mitsubishi notamment) est de fabriquer des robots les plus rigides possibles tout en donnant la possibilité à l'intégrateur de paramétrer le degré de compliance en bout de bras afin de permettre à ce dernier de dériver de sa position cible sans génération d'un arrêt d'urgence (ex : mise en place de goupilles, recul du bras dû aux extracteurs d'une presse à injecter, ...).

3.1.8 Le système de commande

La dernière des 8 caractéristiques est de première importance et malheureusement, elle est beaucoup trop souvent négligée par les intégrateurs. Chaque marque possède sa propre commande à quelques exceptions près. Certaines de ces commandes sont très performantes et ont à disposition notamment les fonctionnalités suivantes:

- Gestion multitâche
- Gestion du protocole TCP/IP via le réseau ethernet
- Gestion des bus de terrain (ASI, Can bus, Interbus-S, Profibus DP, DeviceNet, etc...)
- Software à disposition pour la dite commande
- Possibilité d'intégration d'un système de vision

Toutes ces fonctionnalités doivent absolument être prises en compte et elles peuvent être déterminantes quant au choix du robot. Cependant, il faut garder à l'esprit qu'un intégrateur ne peut intégrer de manière optimale qu'un robot associé à la commande qu'il connaît parfaitement bien.



4 Le bras (partie mécanique)

Le bras est la partie maîtresse du robot, il en existe plusieurs types sur le marché et chacun d'eux est conçu pour des applications bien ciblées, ce qui démontre que le robot industriel est loin d'être une machine universelle. Le module mécanique essentiel du bras est son système de réduction. En effet, les moteurs utilisés actuellement n'ont pas assez de couple pour permettre une transmission directe (direct drive). Il faut ajouter entre le moteur et l'axe à mettre en mouvement un système de réduction. Le choix de ce dernier influencera considérablement les caractéristiques physiques du robot. Chaque fabricant utilise des systèmes de réduction sans jeux tel que l'harmonique drive ou des systèmes de réduction par courroie crantée. Les meilleurs robots du marché ont des systèmes de réduction qui font souvent l'objet d'un brevet (comme le Joint Combiné Stäubli de Stäubli).



Réducteur planétaire avec moteur



JCS de Stäubli

4.1 Les types d'architectures

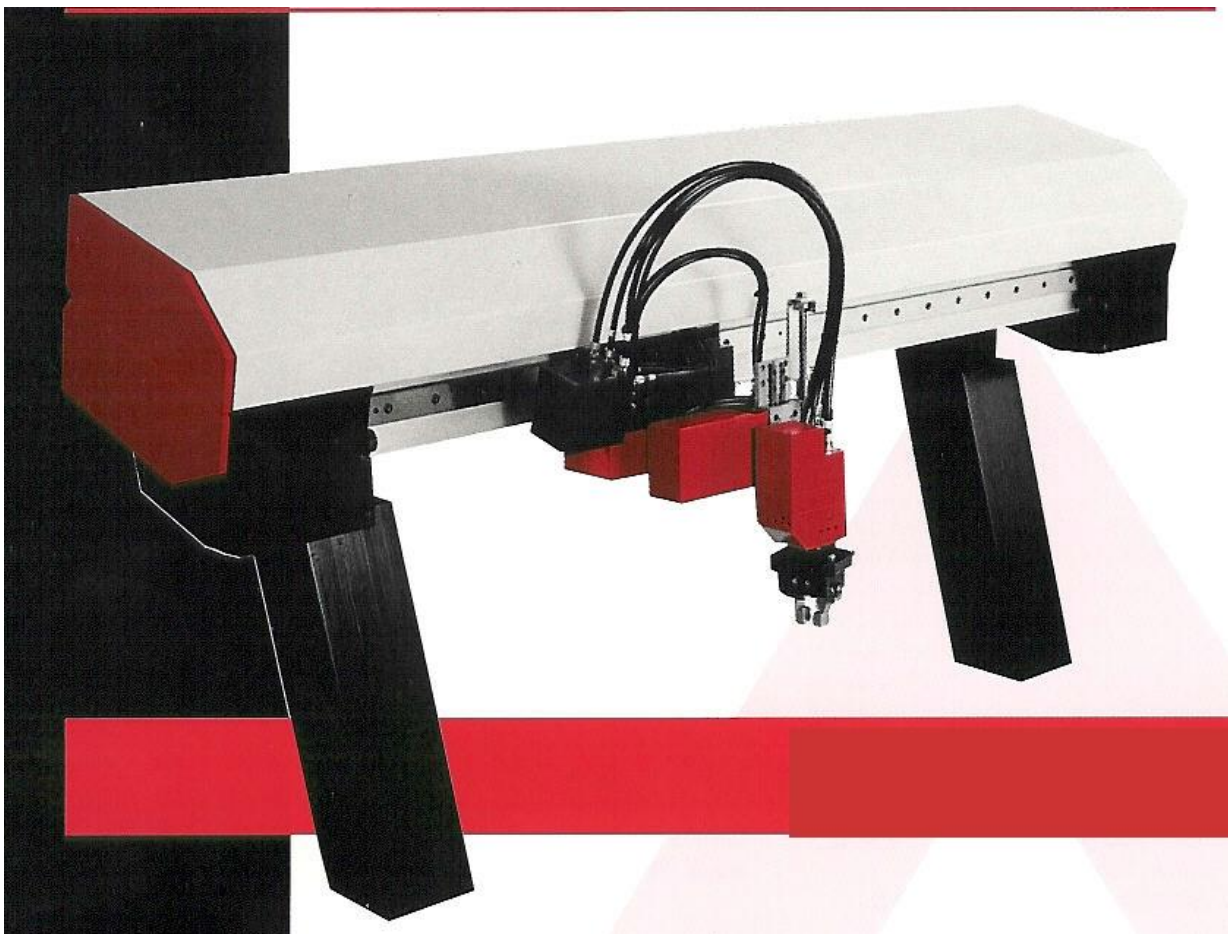
Il existe actuellement un nombre important de types de bras, comprenant chacun un nombre d'axes différents. Il est par conséquent intéressant d'effectuer une classification. Celle qui vous est présentée dans ce cours est une possibilité, la littérature vous propose certainement d'autres variantes.

4.2 L'architecture Série

4.2.1 Les robots autoportiques ou cartésiens

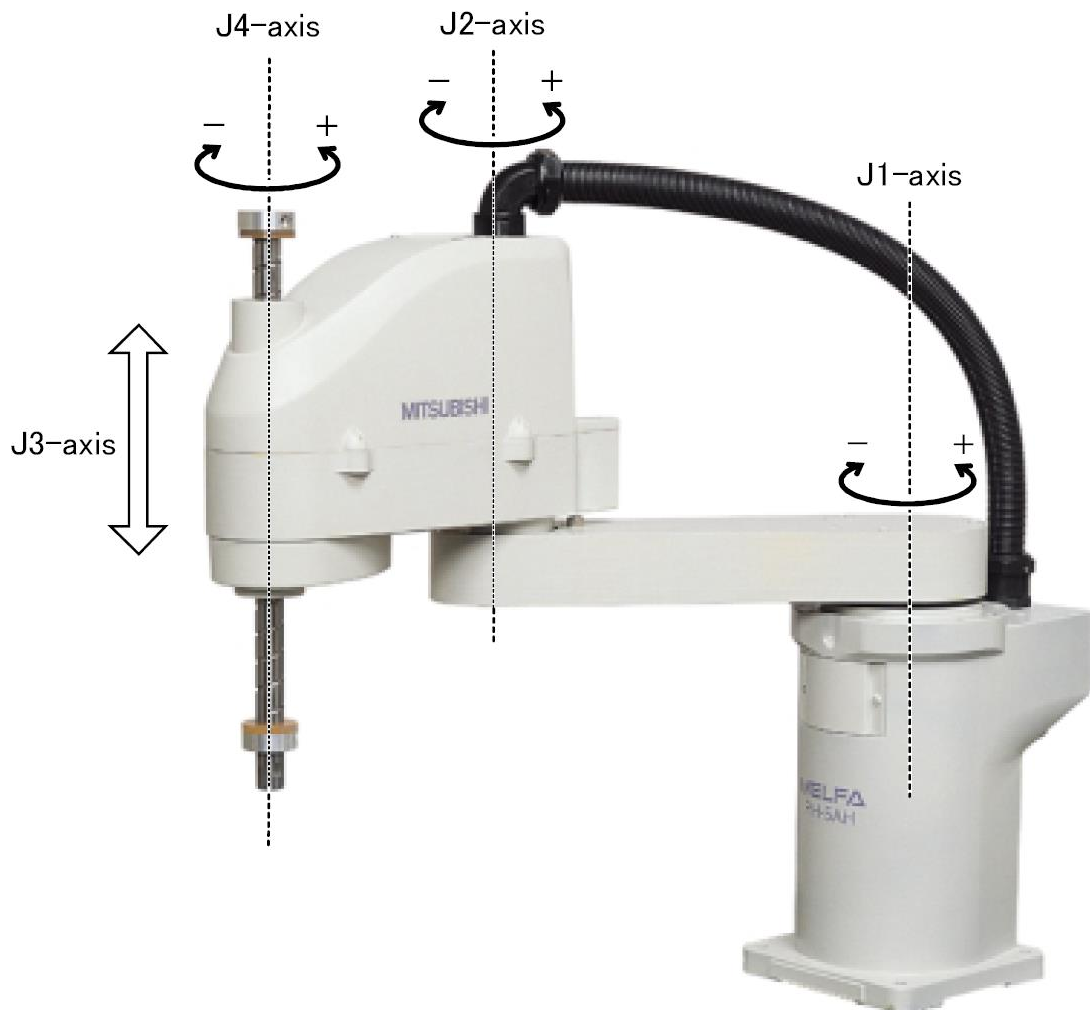
Ces robots (*voir figure ci-dessous*) ont une conception relativement proche des centres d'usinage à commande numérique. Ils sont conçus selon une succession d'axes de translation. Ces robots ont l'avantage d'être précis et rapide cependant, leur enveloppe de travail est relativement restreinte.

Caractéristiques des portiques (ex: sysmelec AUTOPLACE 420)	
Nombre de degré de liberté	4 (3 axes de translation et 1 axe de rotation)
Répétabilité	± 0.005 mm
Enveloppe de travail	Parallélépipède rectangle
Charge transportable admise	4 kg
Vitesse maximum	1,2 m/s



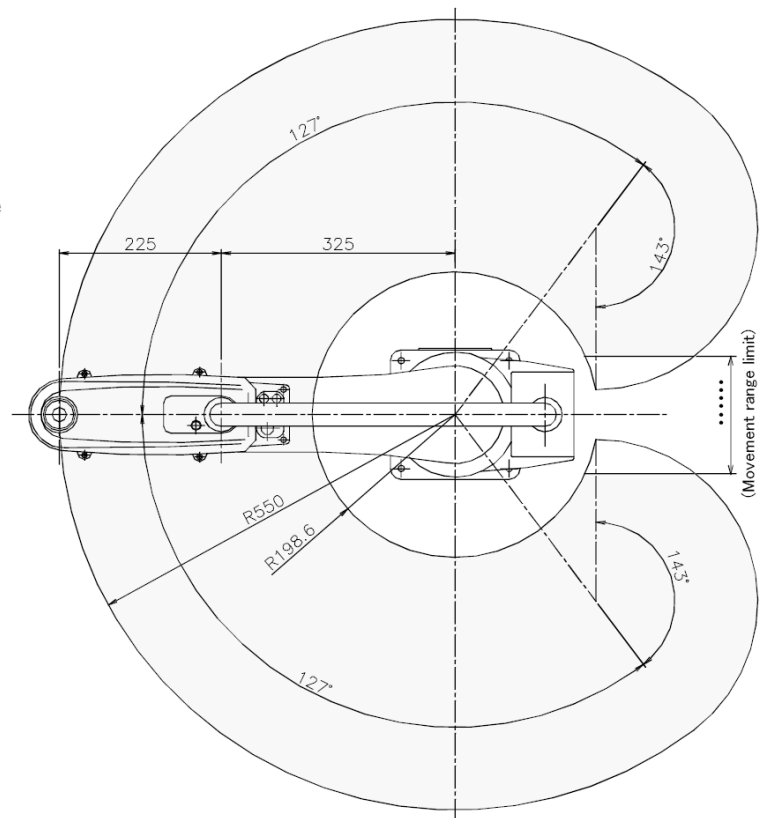
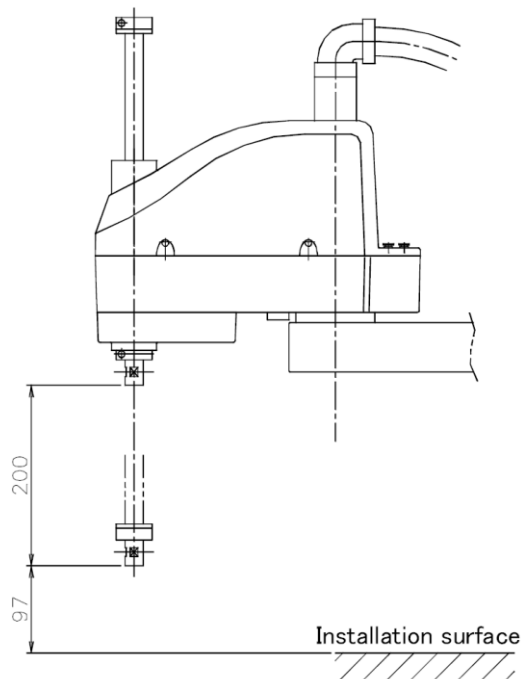
4.2.2 Les robots SCARA

Le mot SCARA vient de l'anglais et signifie "**S**elective **C**ompliance **A**ssembly **R**obot **A**rm" (voir figure ci-dessous) cela signifie que le bras possède une position d'équilibre. Par conséquent, il ne s'effondre pas sous son propre poids. Ces robots sont rapides et ont une bonne répétabilité. Ils excellent dans les applications de pick and place.



La figure ci-dessus montre clairement la cinématique des 4 articulations. Le joint 1, le joint 2 ainsi que le joint 4 génèrent des mouvements de rotation tandis que le joint 3 génère un mouvement de translation. L'enveloppe de préhension est facile à visualiser. Il faut savoir que lors de l'utilisation d'un robot SCARA, toutes les pièces à manipuler doivent impérativement se trouver dans une suite de plans parallèles les uns aux autres.

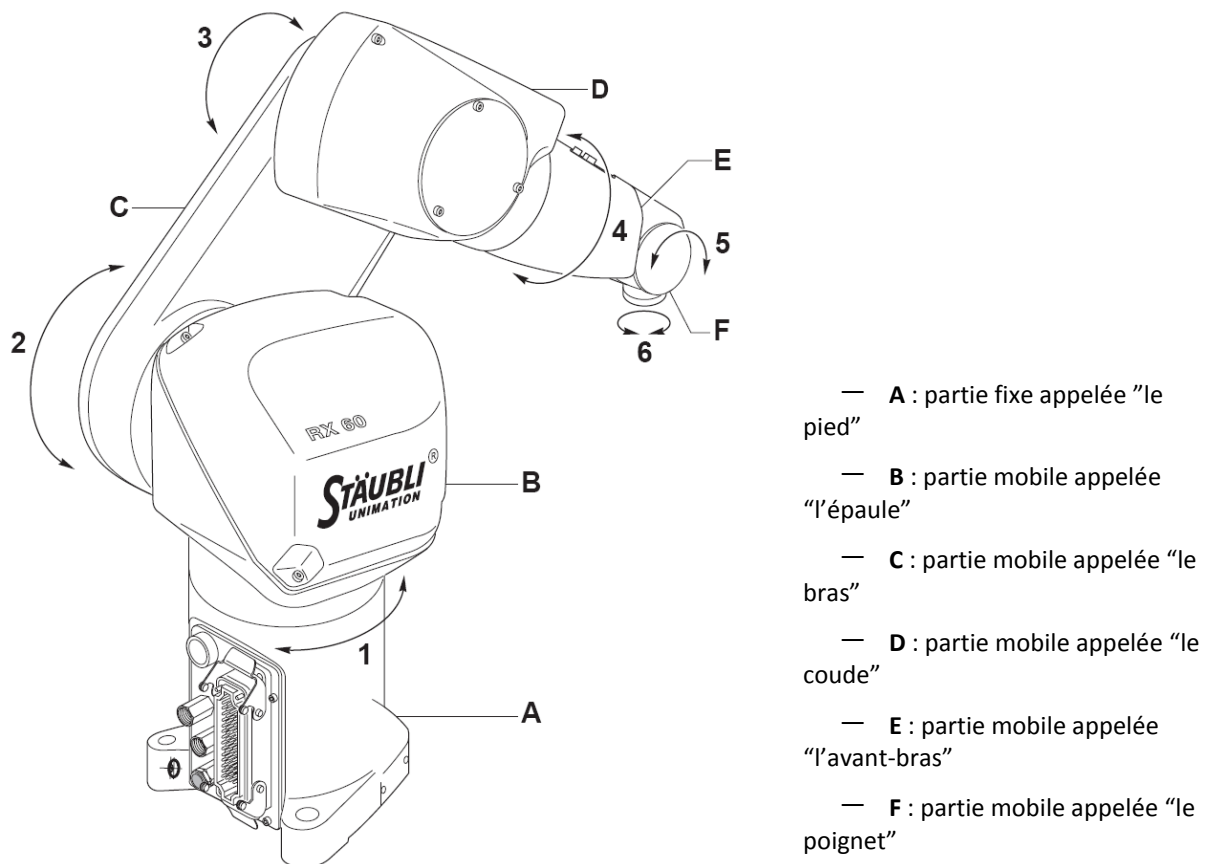
Caractéristiques des scaras (ex: Mitsubishi RH-5AH55)	
Nombre de degré de liberté	4 (1 axe de translation et 3 axes de rotation)
Répétabilité	± 0.02 mm en XY, ± 0.01 mm en Z et $\pm 0.03^\circ$
Enveloppe de travail (voir figure 6)	Cylindre creux
Charge transportable admise	5 kg
Vitesse maximum	Joint 1: $337^\circ/\text{s}$, Joint 2: $540^\circ/\text{s}$, Joint 3: 1000 mm/s, Joint 4: $1870^\circ/\text{s}$



4.2.3 Les robots anthropomorphes

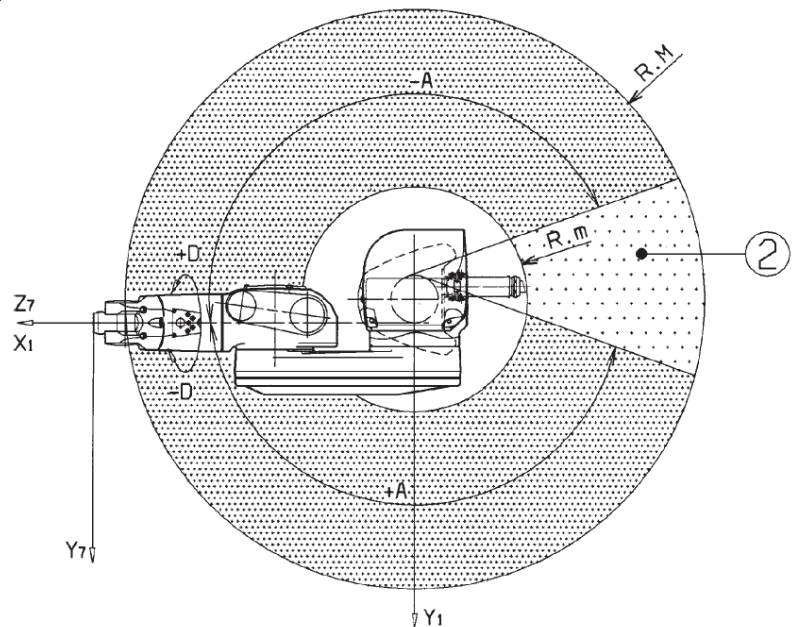
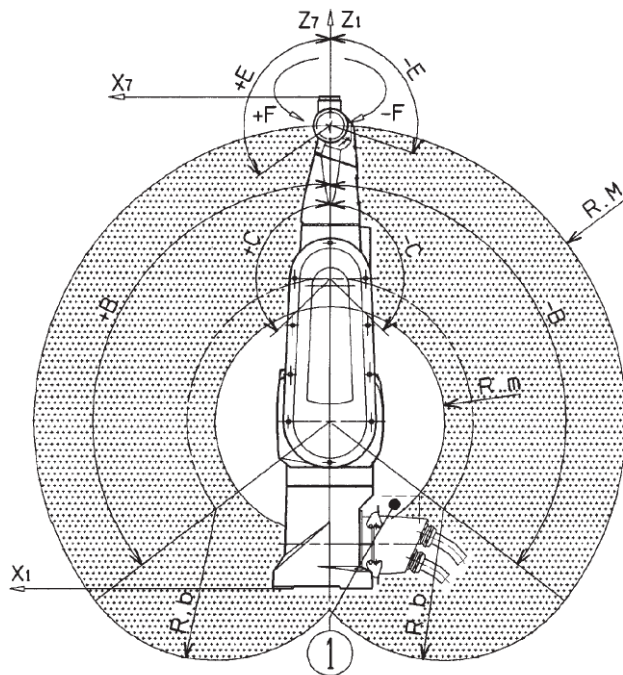
Le mot “anthropomorphe” signifie que la forme rappelle celle de l’Homme. Par conséquent, ces robots sont plus polyvalents que les SCARA, Ils sont capables d’effectuer le 80% des applications que du marché. Cependant, la performance des mouvements du bras leur fait payer un lourd tribut au niveau de la vitesse de déplacement. C’est dans la conception de ce style de bras regroupe la plus grande concentration de marques, cela n’a rien de surprenant car la grande majorité des applications du marché est destinée à ce genre de machine.

Les robots anthropomorphes possèdent entre 5 et 6 degrés de liberté et ont une sphère creuse comme enveloppe de travail (*voir figure ci dessous*).



La figure ci-dessus montre bien qu’il y a une analogie avec l’anatomie humaine, d’où le nom de “robots anthropomorphes”.

Caractéristiques des anthropomorphes (ex: Stäubli RX 60)	
Nombre de degré de liberté	6 (6 axes de rotation)
Répétabilité	± 0.02 mm
Enveloppe de travail	Sphère creuse
Charge transportable admise	2.5 kg
Vitesse maximum	Joint 1: 287°/s, Joint 2: 287°/s, Joint 3: 319°/s, Joint 4: 410°/s, Joint 5: 320°/s, Joint 6: 700°/s



4.3 L'architecture hybride

Tous les autres types de robots sont arbitrairement mis dans cette catégorie qui ne représente que moins de 5% du marché. Il s'agit de robots ayant des caractéristiques propres ce qui leur confère des particularités techniques intéressantes pour des applications spécifiques.

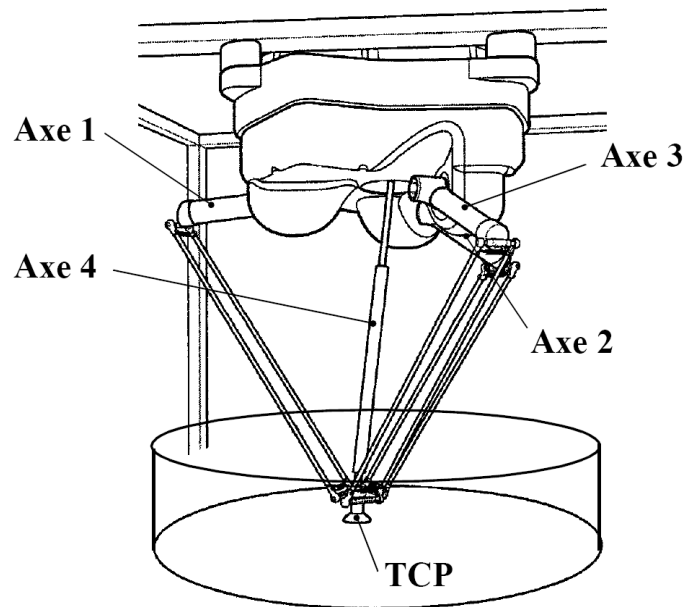
4.3.1 Le robot delta ABB IRB 340

Dans le but de diminuer les masses en mouvement, les moteurs sont ramenés au niveau de la base du robot et les structures en mouvements (les bras) sont fabriquées en fibre de carbone. Pour augmenter encore la rigidité, les bras sont doublés. Ce robot a l'avantage d'avoir une cadence très élevée (plus de 120 pièces/minute) au détriment de sa répétabilité et de son enveloppe de travail réduite.

Ce type de robot est principalement utilisé dans l'industrie alimentaire.



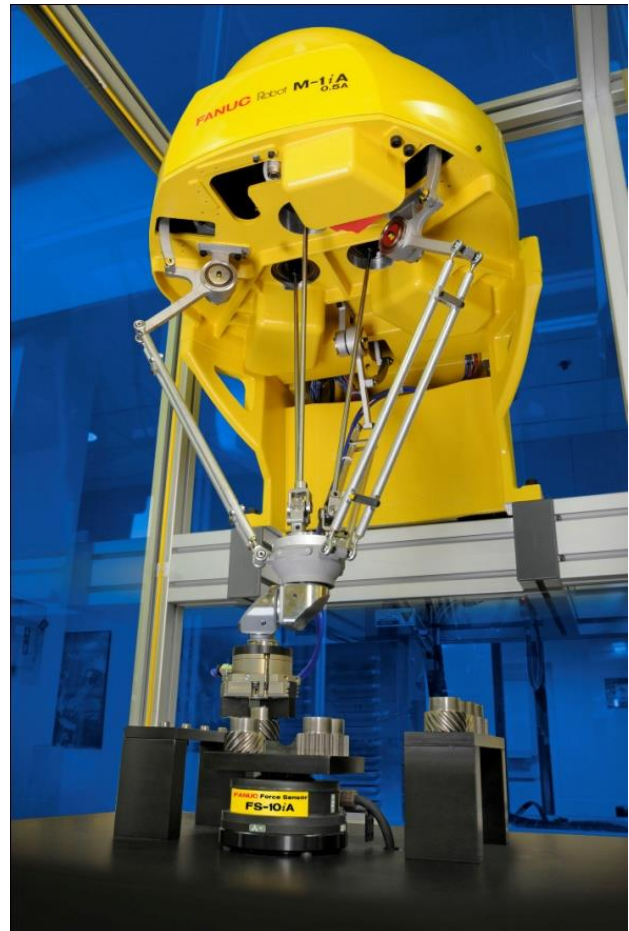
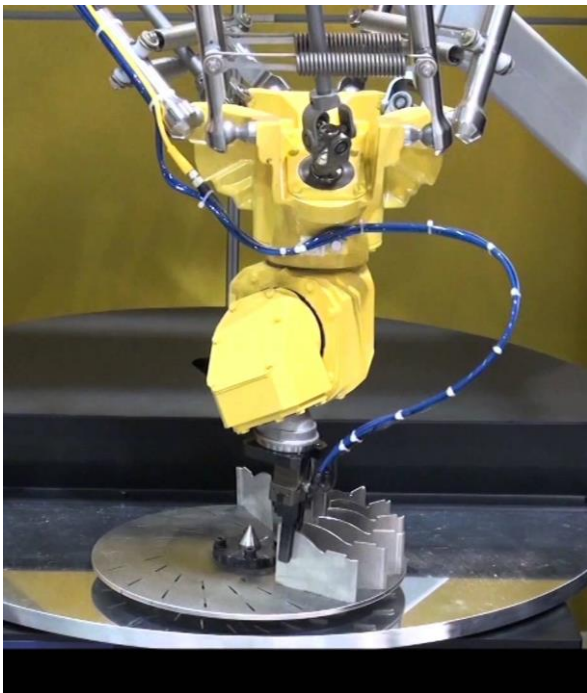
Figure 1 ABB IRB 340



Caractéristiques du robot Delta (ex: ABB IRB 340)	
Nombre de degré de liberté	4 (4 axes de rotation)
Répétabilité	± 0.1 mm en XYZ et 0.4° en rotation
Enveloppe de travail	Cylindrique
Charge transportable admise	1 kg
Vitesse maximum	X, Y, Z: 10 m/s, $\dot{\theta}$: 3600°/s

4.3.2 Le robot Fanuc M-1iA

Le constructeur japonais Fanuc propose un robot delta qui associe une structure parallèle pour la translation (X-Y-Z), ainsi qu'un poignet avec 3 axes rotatifs pour la rotation (RX, RY et RZ). La transmission pour ces 3 axes est faite par des arbres et cardans, de manière à ce que la vocation du robot delta soit respectée: un minimum de masse en mouvement pour une dynamique élevée.



Robot delta Fanuc 6 axes

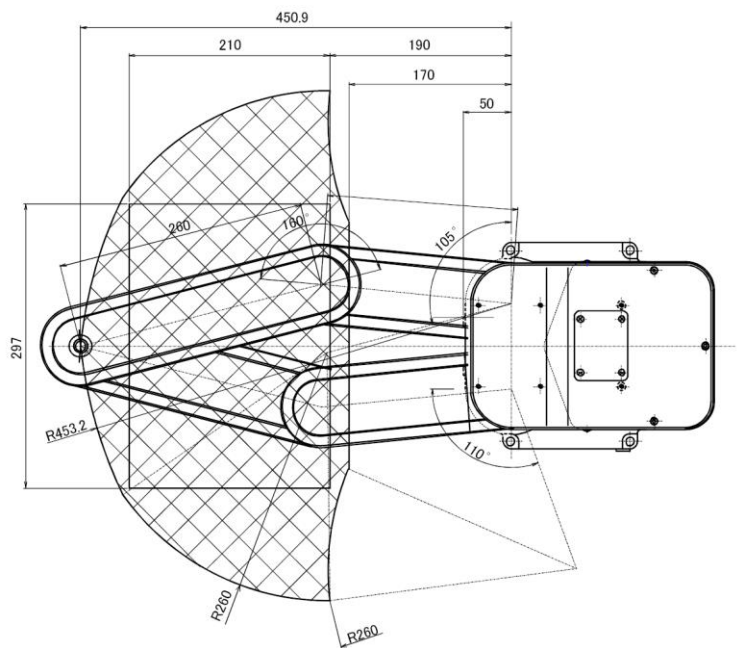
4.3.3 Le robot Mitsubishi de type RP-5AH

Dans ce cas également, les moteurs 1 et 2 sont ramenés sur la base du robot ce qui assure une excellente dynamique. La rigidité est accrue par la mise en parallèle des bras ce qui permet d'obtenir une excellente répétabilité au détriment d'une enveloppe de travail confortable. Son domaine de prédilection est l'assemblage de composants microtechniques.

Caractéristiques des SCARA parallèles (ex: Mitsubishi RP-5AH)	
Nombre de degré de liberté	4 (1 axe de translation et 3 axes de rotation)
Répétabilité	± 0.01 mm en XY, ± 0.01 mm en Z et $\pm 0.03^\circ$ en rotation
Enveloppe de travail (voir figure 6)	« Demi Cylindre »
Charge transportable admise	5 kg
Vitesse maximum	Joint 1: 432°/s, Joint 2: 432°/s, Joint 3: 960 mm/s, Joint 4: 1330°/s



Figure 2 Mitsubishi RP



4.3.4 L'hexapode

L'hexapode est un autre type de robot 6 axes parallèles. L'amplitude de ses mouvements est relativement restreinte, mais il possède une rigidité très élevée.



Figure 3 Hexapode

4.4 Les robots coopératifs

L'évolution de la gestion des mouvements ainsi que de l'entraînement des robots permet de créer des robots pouvant interagir avec l'homme, sans enceinte de sécurité:

- faible inertie grâce à une construction allégée du bras.
- vitesse tangentielle faible ($< 250 \text{ mm/s}$).
- contrôle de la force ($< 150 \text{ N}$) : détection de contraintes, collision, via la mesure du couple de chaque axe.
- compensation de la gravité.
- apprentissage coopératif : pour les phases d'apprentissage, l'opérateur déplace le robot en le guidant physiquement. La mesure de couple est suffisamment sensible pour que le robot détecte le mouvement qui lui est demandé.



Figure 4 Kuka LBR5



Figure 5 Universal Robot

5 La commande (partie électronique)

La commande est le deuxième élément fondamental qui compose le robot. Elle est munie d'un microprocesseur, de cartes d'axes, d'amplificateurs et de cartes qui permettent de dialoguer avec le monde extérieur (RS-232, TCP/IP, I/O, Devicenet, Profibus DP, ...). Les performances de la paire « commande – robot » et la fiabilité du système d'exploitation sont déterminants pour la réussite d'une application. La majorité des constructeurs développent leur propre commande pour exploiter au mieux les performances mécaniques du bras robotique. Une bonne commande doit posséder les caractéristiques suivantes :

- Système d'exploitation stable et performant
- Bonne régulation des mouvements du bras (répétabilité et précision, gestion des vitesses et accélérations, pas d'overshoot)
- Approche du « temps réel »
- Communication avec les éléments péri-robotiques (IO digitales, gestion des bus de terrain)
- Gestion des protocoles de communication standards (RS 232, TCP/IP, etc...)
- Gestion performante des fichiers à travers un réseau (NFS, FTP, etc...)
- Gestion du multitâche
- Possibilité d'interfaçage du TeachPendant
- Accès aux fonctions par Activex

6 Systèmes de coordonnées

La tâche fondamentale d'un robot est de se positionner dans l'espace, selon une *translation* et une *orientation* donnée. Deux systèmes de coordonnées différents sont utilisés :

1. Le système de coordonnées **articulaire** (*joint*), système de coordonnées natif du robot industriel. Les articulations sont de type angulaire (rotation) ou linéaire (translation). Une position articulaire permet de définir la position de chaque axe mécanique.
2. Le système de coordonnées **cartésien**. Deux repères cartésiens principaux sont définis par défaut, l'un fixe, à la base du robot (repère **world**), l'autre mobile, à l'extrémité de l'outil du robot (repère **tool**). Celui-ci est fixé sur la *flasque* du robot, le $(n+1)^{\text{ème}}$ segment, situé lui-même après l'axe n .

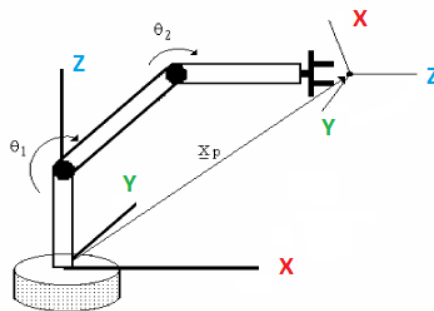
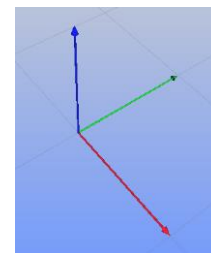


Figure 1. Base Coordinate system.

Remarques :

- le repère cartésien est toujours orthonormé
- X en rouge, Y en vert et Z en bleu !



6.1 Calcul direct et inverse

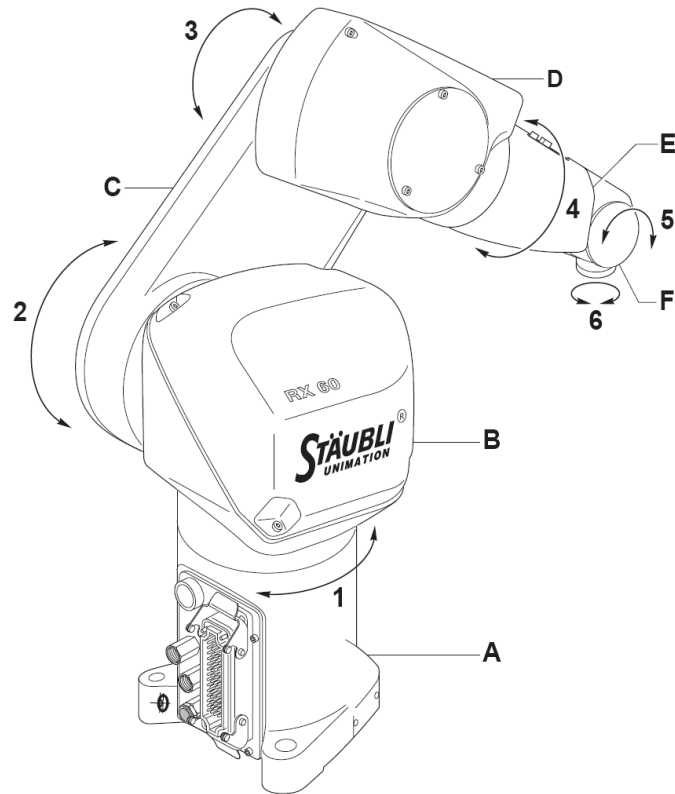
La calcul directe permet de calculer la position dans le système *cartésien* (world) correspondant à une position articulaire donnée. Le résultat correspond à la multiplication des matrices de transfert de chaque axe du robot (selon la convention de Denavit-Hartenberg).

Pour une position articulaire donnée, il n'existe qu'une seule position cartésienne.

La calcul inverse permet de calculer la ou les positions articulaires correspondant à une position cartésienne donnée. Les cas où les solutions sont multiples correspondent aux différentes *configurations* possibles du bras (possible pour les architectures série et parallèle).

Pour une position cartésienne donnée, il peut exister plusieurs positions articulaires.

6.2 Le système de coordonnées articulaire

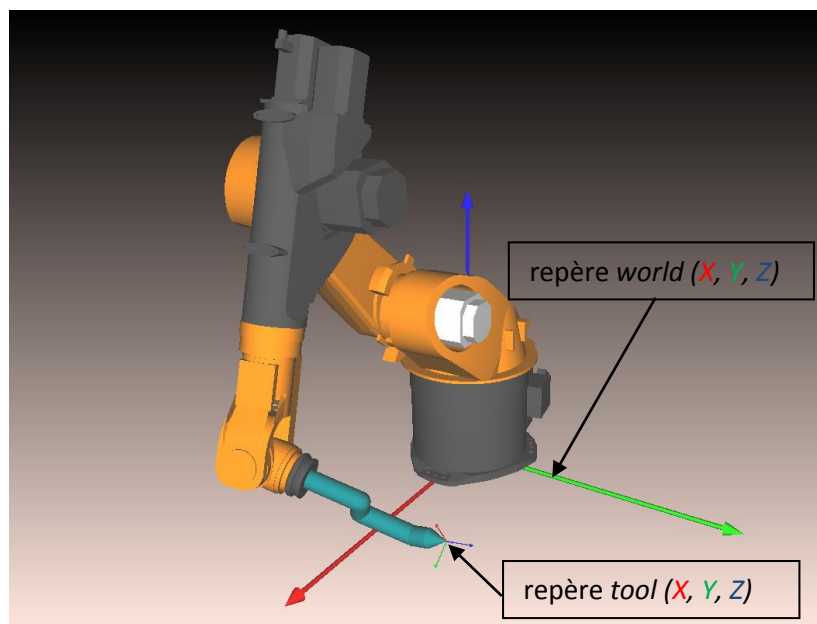


Le système articulaire définit une position mécanique pour chaque axe composant le bras, selon une valeur en $[\circ]$ pour les articulations angulaires et en $[\text{mm}]$ pour les articulations linéaires. Il s'agit d'une valeur brute directement générée par les codeurs. Ces n valeurs, correspondant aux n degrés de liberté du bras, définissent non seulement une position géographique du système de préhension mais également une configuration physique du bras. La combinaison de ces n valeurs définit une position **unique** du bras et une configuration donnée.

6.3 Le système de coordonnées cartésien

Le système articulaire peut être difficile à visualiser lorsque qu'il faut interpréter des mouvements rectilignes ou des déplacements relatifs en [mm], notamment dans le cas des robots à articulation angulaires. Afin de pouvoir s'affranchir du système articulaire et de réaliser aisément les différentes tâches qui leur incombent, les robots industriels possèdent un deuxième système de coordonnées, le système *cartésien*. Celui-ci définit deux repères principaux :

1. le repère *world*, fixe, à la base du robot, avant le 1^{er} axe mécanique.
2. le repère *tool*, mobile, à l'extrémité de l'outil/préhenseur monté sur le robot, celui-ci étant fixé sur la *flasque* du robot, le $(n+1)^{\text{ème}}$ segment, situé lui-même après l'axe n .



les repères *world* et *tool* du robot

Remarques :

- la transformée géométrique du préhenseur, appelée également le **tool** du robot, correspond à la position du repère de l'outil/préhenseur monté sur le robot par rapport à la flasque (extrémité mécanique) du robot. Cette transformée est additionnée à la matrice de transfert du dernier axe (voir Denavit-Hartenberg) et est considérée pour tous les calculs *directs* et *inverse*.
- un **tool nul** indique que le robot est utilisé sans outillage/préhenseur monté sur sa flasque.
- le repère *tool* étant mobile, celui-ci n'est pas utilisé pour définir une position fixe autour du robot. Il est généralement utilisé pour les déplacements manuels lors de l'apprentissage de position dans l'environnement du robot.
- lors de la lecture de la position cartésienne du bras, c'est toujours par rapport au repère *world* que celle-ci est exprimée.
- exécuter un mouvement cartésien revient à demander au contrôleur du robot de positionner le robot de manière à ce que le repère *tool* du robot vienne coïncider avec une position cartésienne exprimée par rapport au repère *world*.
- par convention, l'axe Z du repère *tool* est toujours dans le prolongement de l'outil.
- un préhenseur/outil peut comporter plusieurs extrémités de travail, correspondant à autant de transformées *tool*.

6.3.1 Système cartésien - les matrices de positions

Six coordonnées sont nécessaires pour définir une position cartésienne *secondaire* dans l'espace par rapport à un repère *principal*:

3 translations et 3 rotations

- 3 translations correspondent aux coordonnées X, Y et Z.
- 3 angles de rotation. Ceux-ci peuvent être définis de différentes manières, mais il faut au minimum une rotation autour de chacun des axes principaux

Les matrices permettent de regrouper en un seul objet ces 6 variables, et en facilitent les différentes opérations (multiplication, inversion, etc).

6.3.1.1 Matrice de translation

La matrice de translation exprime la translation d'un point selon les axes X, Y et Z du repère *primaire*.

$$Tr := \begin{pmatrix} dx \\ dy \\ dz \end{pmatrix}$$

6.3.1.2 Matrices de rotation

La matrice de rotation exprime la rotation en décrivant les 3 vecteurs unitaires (x, y, z) du système d'axes orthonormé *secondaire* (matrice 3x3, 9 valeurs), ce qui implique une "redondance" d'information (système orthonormé). Il est possible d'exprimer n'importe quelle rotation dans l'espace tridimensionnel par 3 ou 4 valeurs. Voici les principales méthodes existantes pour *lire* et *écrire* une même matrice de rotation :

- angles d'*Euler* : rotations successives Rz, Rx, Rz
- angles de *Cardan* Yaw-pitch-roll : rotations successives Rz, Ry, Rz
- RX-RY-RZ : rotations successives Rx, Ry, Rz
- *Quaternions* : rotation d'un angle donné autour d'un vecteur: 1 + 3 = 4 coordonnées

Remarque: rotations successive signifie que chaque rotation est faite par rapport au système modifié par les rotations précédentes.

$$R := \begin{pmatrix} x_x & y_x & z_x \\ x_y & y_y & z_y \\ x_z & y_z & z_z \end{pmatrix}$$

$$RX(rx) := \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(rx) & -\sin(rx) \\ 0 & \sin(rx) & \cos(rx) \end{pmatrix} \quad RY(ry) := \begin{pmatrix} \cos(ry) & 0 & \sin(ry) \\ 0 & 1 & 0 \\ -\sin(ry) & 0 & \cos(ry) \end{pmatrix} \quad RZ(rz) := \begin{pmatrix} \cos(rz) & -\sin(rz) & 0 \\ \sin(rz) & \cos(rz) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

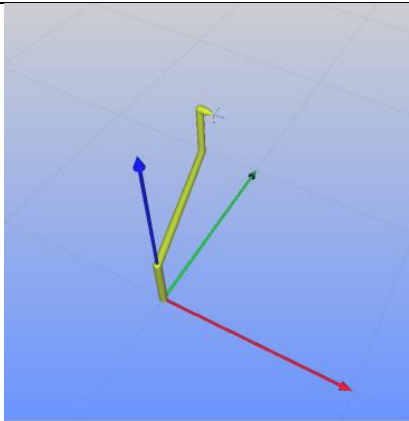
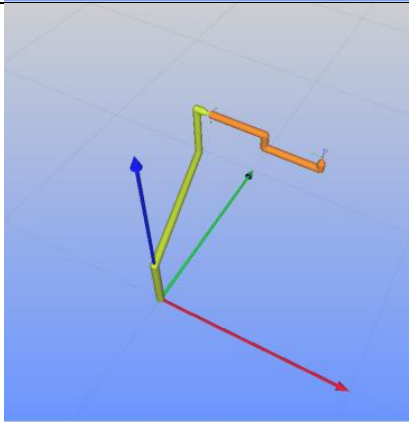
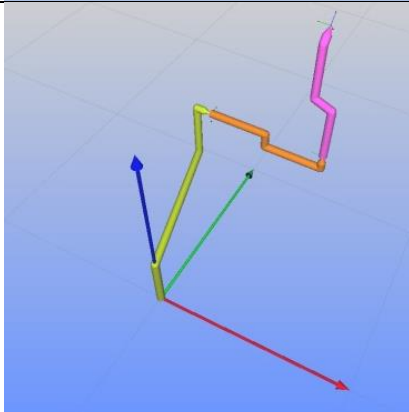
6.3.1.3 Matrice de position

Elle regroupe une matrice de *translation* (3x1) et une matrice de *rotation* (3x3) en une seule matrice. Elle est complétée par une 4^{ème} ligne (0,0,0,1) de manière à obtenir une matrice carrée.

Elle définit la *translation* et la *rotation* d'un système *secondaire* quelconque par rapport à un système *primaire*. Les matrices de positions peuvent être multipliées entre elles afin de calculer une transformée totale. Cette opération est non-commutative.

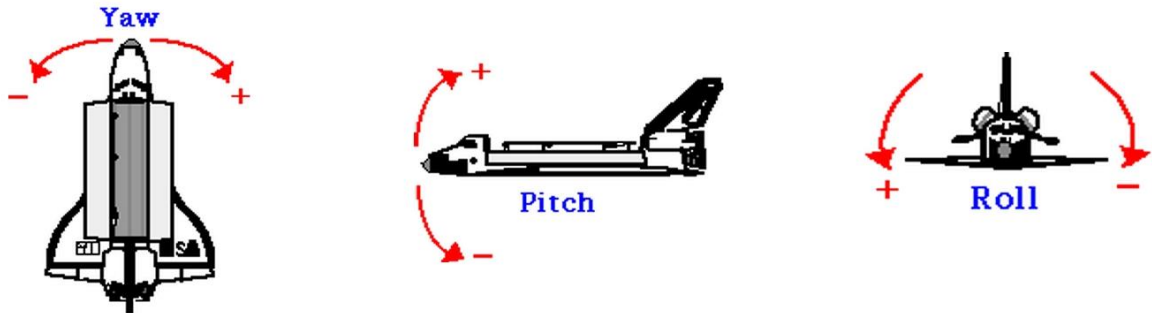
$$P := \begin{pmatrix} \mathbf{R} & \mathbf{Tr} \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} x_x & y_x & z_x & dx \\ x_y & y_y & z_y & dy \\ x_z & y_z & z_z & dz \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Exemple d'addition de transformées (produit matricielle) :

1 transformée		<table><tr><th></th><th><i>x</i></th><th><i>y</i></th><th><i>z</i></th><th><i>rx</i></th><th><i>ry</i></th><th><i>rz</i></th></tr><tr><td>T1</td><td>-120</td><td>720</td><td>600</td><td>0</td><td>90</td><td>0</td></tr></table>		<i>x</i>	<i>y</i>	<i>z</i>	<i>rx</i>	<i>ry</i>	<i>rz</i>	T1	-120	720	600	0	90	0																					
	<i>x</i>	<i>y</i>	<i>z</i>	<i>rx</i>	<i>ry</i>	<i>rz</i>																															
T1	-120	720	600	0	90	0																															
2 transformées		<table><tr><th></th><th><i>x</i></th><th><i>y</i></th><th><i>z</i></th><th><i>rx</i></th><th><i>ry</i></th><th><i>rz</i></th></tr><tr><td>T1</td><td>-120</td><td>720</td><td>600</td><td>0</td><td>90</td><td>0</td></tr><tr><td>T2</td><td>0</td><td>0</td><td>600</td><td>0</td><td>-90</td><td>90</td></tr><tr><td>R</td><td>480</td><td>720</td><td>600</td><td>0</td><td>0</td><td>90</td></tr></table>		<i>x</i>	<i>y</i>	<i>z</i>	<i>rx</i>	<i>ry</i>	<i>rz</i>	T1	-120	720	600	0	90	0	T2	0	0	600	0	-90	90	R	480	720	600	0	0	90							
	<i>x</i>	<i>y</i>	<i>z</i>	<i>rx</i>	<i>ry</i>	<i>rz</i>																															
T1	-120	720	600	0	90	0																															
T2	0	0	600	0	-90	90																															
R	480	720	600	0	0	90																															
3 transformées		<table><tr><th></th><th><i>x</i></th><th><i>y</i></th><th><i>z</i></th><th><i>rx</i></th><th><i>ry</i></th><th><i>rz</i></th></tr><tr><td>T1</td><td>-120</td><td>720</td><td>600</td><td>0</td><td>90</td><td>0</td></tr><tr><td>T2</td><td>0</td><td>0</td><td>600</td><td>0</td><td>-90</td><td>90</td></tr><tr><td>T3</td><td>100</td><td>100</td><td>600</td><td>0</td><td>45</td><td>0</td></tr><tr><td>R</td><td>380</td><td>820</td><td>1200</td><td>-45</td><td>0</td><td>90</td></tr></table>		<i>x</i>	<i>y</i>	<i>z</i>	<i>rx</i>	<i>ry</i>	<i>rz</i>	T1	-120	720	600	0	90	0	T2	0	0	600	0	-90	90	T3	100	100	600	0	45	0	R	380	820	1200	-45	0	90
	<i>x</i>	<i>y</i>	<i>z</i>	<i>rx</i>	<i>ry</i>	<i>rz</i>																															
T1	-120	720	600	0	90	0																															
T2	0	0	600	0	-90	90																															
T3	100	100	600	0	45	0																															
R	380	820	1200	-45	0	90																															

6.3.2 Exemple de rotation: les angles de Cardan - Yaw-Pitch-Roll

Dans la navigation, l'aéronautique ou l'aérospatiale, le principe des angles de Cardan est souvent utilisé. L'orientation angulaire est définie par 3 axes de rotation qui sont : le lacet (yaw), le tangage (pitch) et le roulis (roll). Ces 3 rotations permettent de définir la position de l'engin.

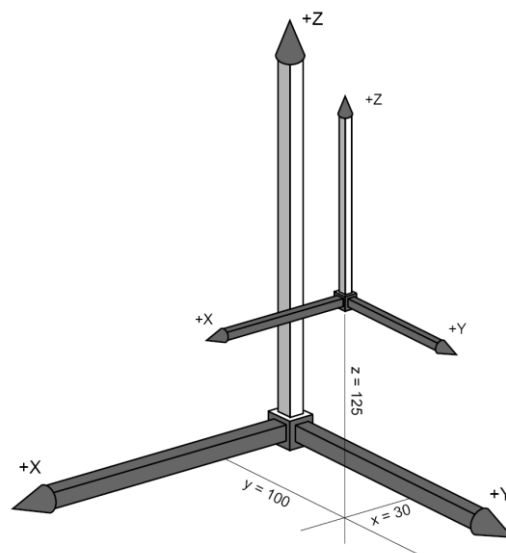


Les robots fonctionnant avec une commande Adept V⁺ utilisent le modèle des angles de Cardan. Par conséquent, la position du système de préhension dans le système cartésien va être définie avec 6 coordonnées : X, Y, Z, yaw, pitch, roll.

Exemples

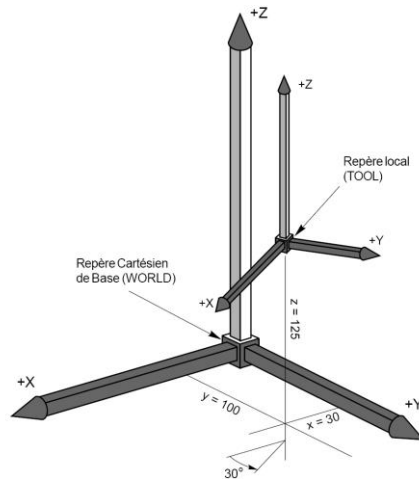
X	Y	Z	yaw	pitch	Roll
30	100	125	0	0	0

L'orientation du repère *secondaire* est identique à l'orientation du repère. Les 3 rotations sont nulles.



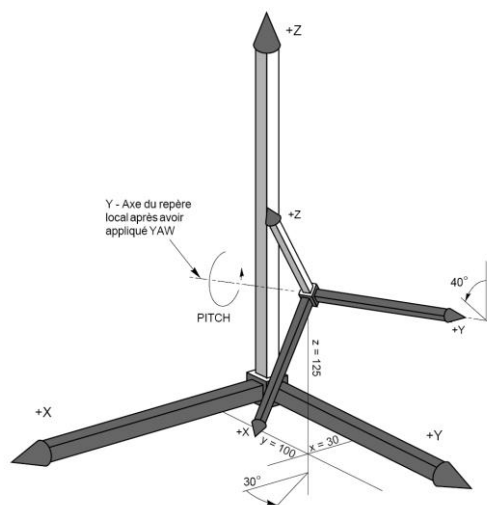
X	Y	Z	yaw	pitch	roll
30	100	125	30	0	0

La rotation **yaw** est une rotation du repère *secondaire* autour de son axe Z.



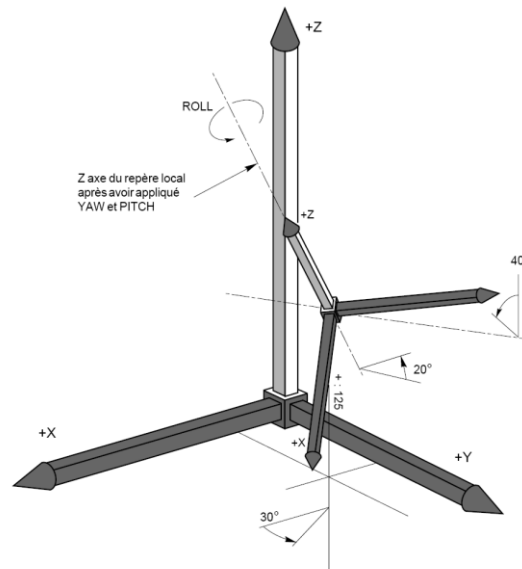
X	Y	Z	yaw	pitch	roll
30	100	125	30	40	0

La rotation **pitch** est une rotation du repère *secondaire* autour de son axe Y, après avoir effectué la rotation **yaw**.



X	Y	Z	yaw	pitch	roll
30	100	125	30	40	20

La rotation **roll** est une rotation du repère *secondaire* autour de son axe Z, après avoir effectué les rotations **yaw** et **pitch**.

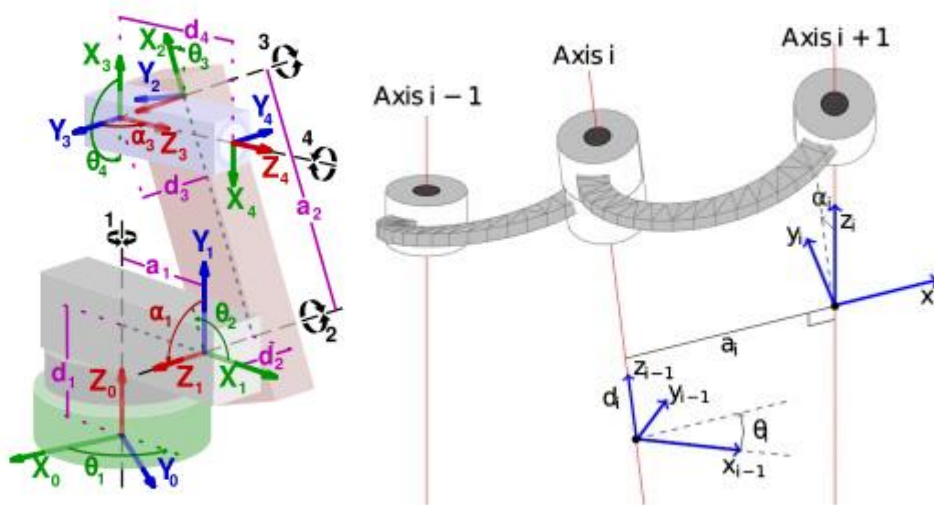


6.4 La méthode Denavit-Hartenberg

Denavit et Hartenberg (D-H) est une convention habituellement utilisée pour choisir le système de référence en robotique. Elle fut introduite en 1955 par Jacques Denavit et Richard S. Hartenberg. Selon cette convention, chaque transformation (passage du trièdre n au trièdre $n+1$) est représentée comme le produit de quatre transformations basiques.

Pour définir ces transformations, il est tout d'abord nécessaire de définir les axes des liaisons :

1. Les axes $\vec{z}_n$ sont suivant les axes des liaisons
2. Les axes $\vec{x}_n$ sont parallèles à la normale commune à $\vec{z}_{n-1}$ et $\vec{z}_n$ soit : $\vec{x}_n = \vec{z}_{n-1} \wedge \vec{z}_n$
3. Les axes $\vec{y}_n$ sont choisis de manière à former un trièdre direct avec les axes $\vec{z}_n$ et $\vec{x}_n$



Exemple de la chaîne cinématique d'un robot avec système de coordonnées et paramètres selon Denavit&Hartenberg.

Chaque transformation entre deux corps successifs est donc décrite par quatre paramètres :

1. d , la distance selon l'axe $\vec{z}_n$ entre les axes $\vec{x}_n$ et $\vec{x}_{n+1}$
2. θ , l'angle entre autour de l'axe $\vec{z}_n$ entre les axes $\vec{x}_n$ et $\vec{x}_{n+1}$
3. A (ou r), la distance selon l'axe $\vec{x}_n$ entre les axes $\vec{z}_{n-1}$ et $\vec{z}_n$. C'est donc également la longueur de la normale commune.
4. α , l'angle entre autour de l'axe $\vec{x}_n$ entre les axes $\vec{z}_{n-1}$ et $\vec{z}_n$

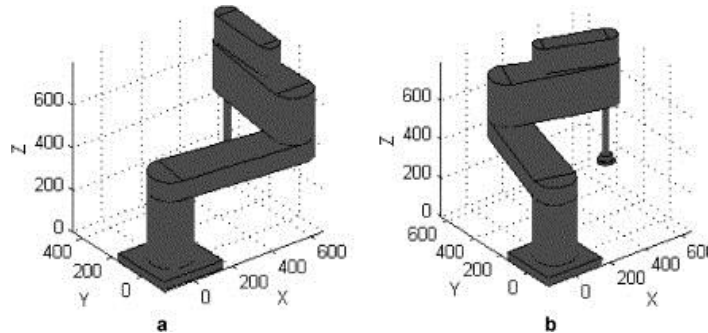
En multipliant les matrices de rotation et de translation élémentaire, la transformation globale entre deux liaisons successives devient :

$$T = \begin{pmatrix} \cos(\theta) & -\cos(\alpha)\sin(\theta) & \sin(\alpha)\sin(\theta) & r\cos(\theta) \\ \sin(\theta) & \cos(\alpha)\cos(\theta) & -\cos(\theta)\sin(\alpha) & r\sin(\theta) \\ 0 & \sin(\alpha) & \cos(\alpha) & d \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Source: <http://fr.wikipedia.org/wiki/Denavit-Hartenberg>

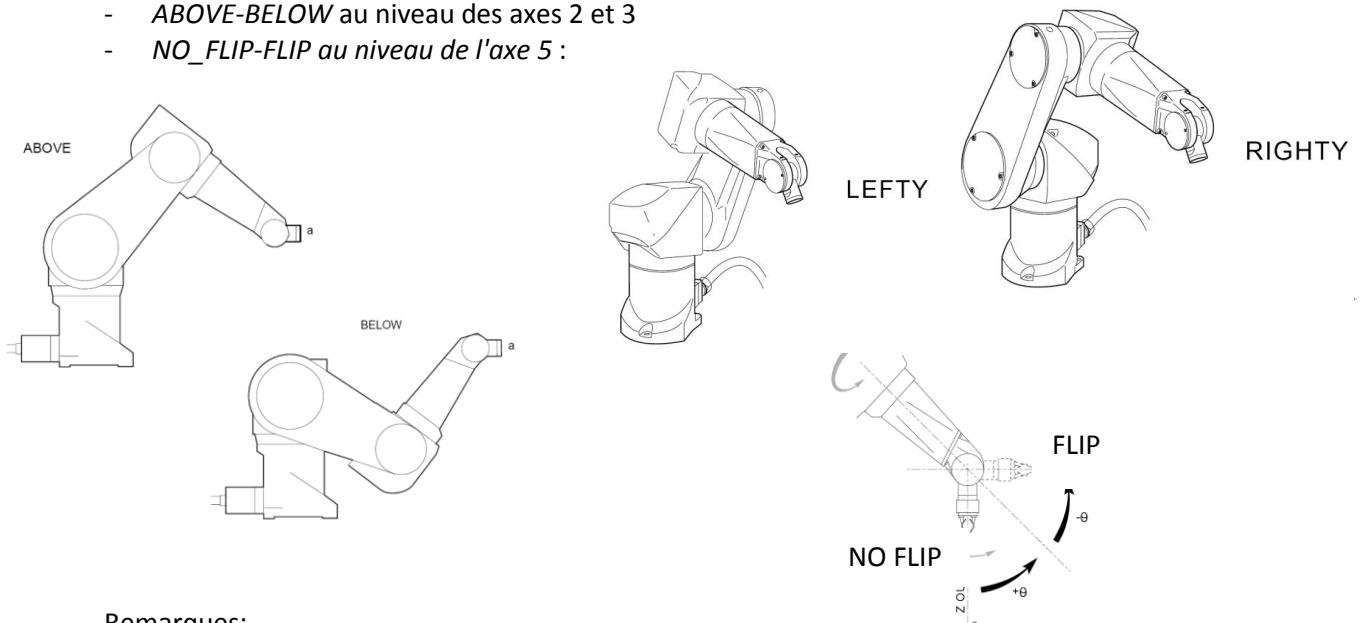
6.5 La configuration du bras

De part leur conception mécanique, les bras robots peuvent atteindre une même position cartésienne de plusieurs manières, appelées *configuration physiques*. Un robot SCARA 4 axes possède une seule paire d'états définissant sa configuration, au niveau des axes 1 et 2, et pouvant prendre la valeur *LEFTY* ou *RIGHTY*.



Un robot anthropomorphe 6 axes possède 3 paires d'états définissant sa configuration :

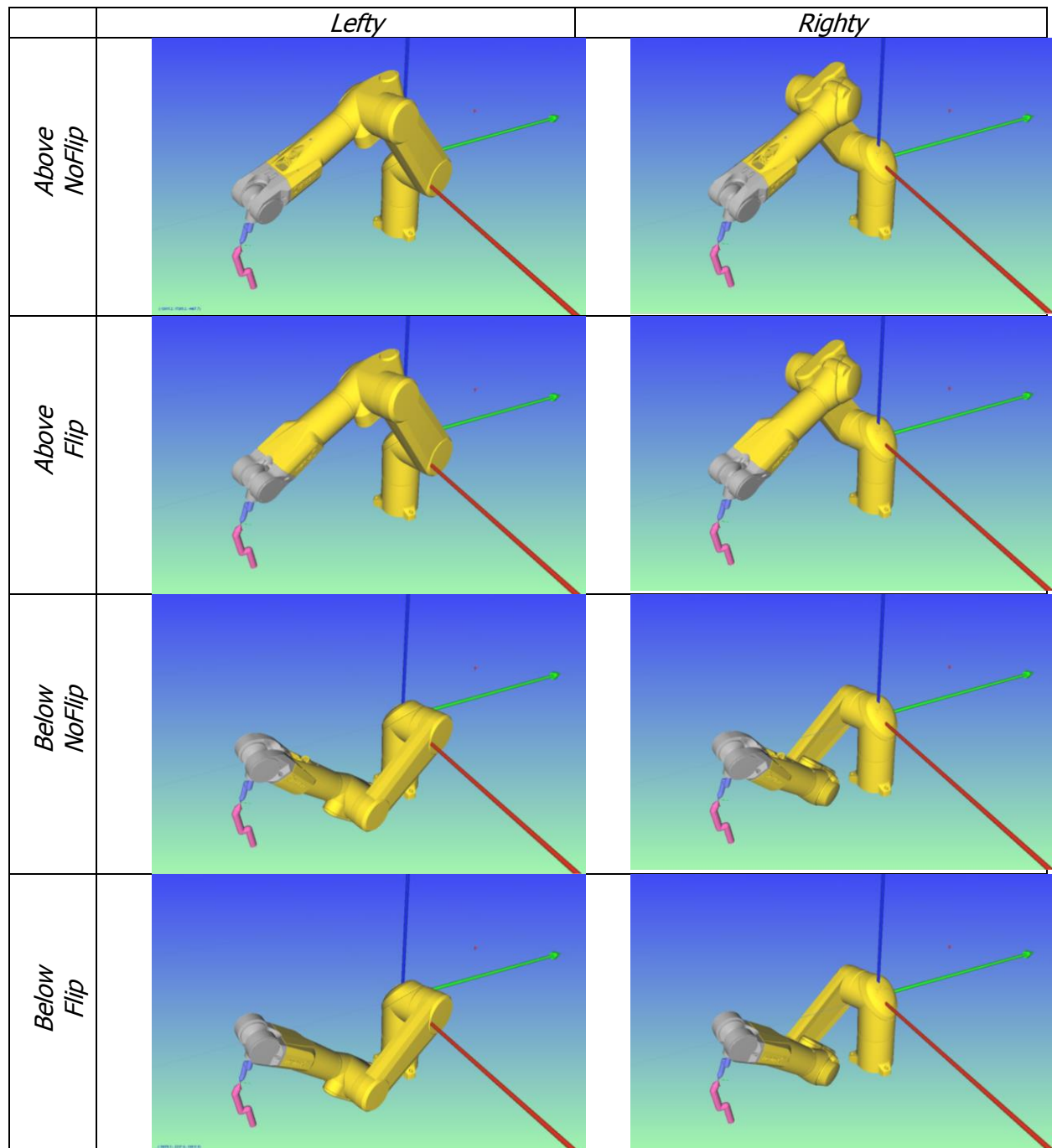
- *LEFTY-RIGHTY* au niveau des axes 1, 2 et 3
- *ABOVE-BELOW* au niveau des axes 2 et 3
- *NO_FLIP-FLIP* au niveau de l'axe 5 :



Remarques:

- lorsqu'un robot est en position alignée (partiellement ou totalement), sa configuration est *ambigüe*, car un ou plusieurs de ses axes sont à la limite de leurs différents états de configuration. Il ne faut jamais effectuer de mouvement cartésien à partir d'une telle position (vitesses et accélérations importantes, voir hors de contrôle => instabilité !).
- la configuration du bras est définie lors d'un mouvement en coordonnées *articulaires*.
- **avant d'effectuer des mouvements *cartésiens*, le robot doit être amené dans ses différentes zones de travail par un mouvement *articulaire* afin d'imposer sa configuration de manière non-ambigüe !**
- l'exécution d'un mouvement cartésien ne modifie pas la *configuration* du bras. Si cela n'est pas imposé par le contrôleur lui même, il est conseillé de respecter cette règle afin d'éviter un changement de configuration pendant un mouvement cartésien. Synaxis impose une configuration constante au cours d'un mouvement cartésien.

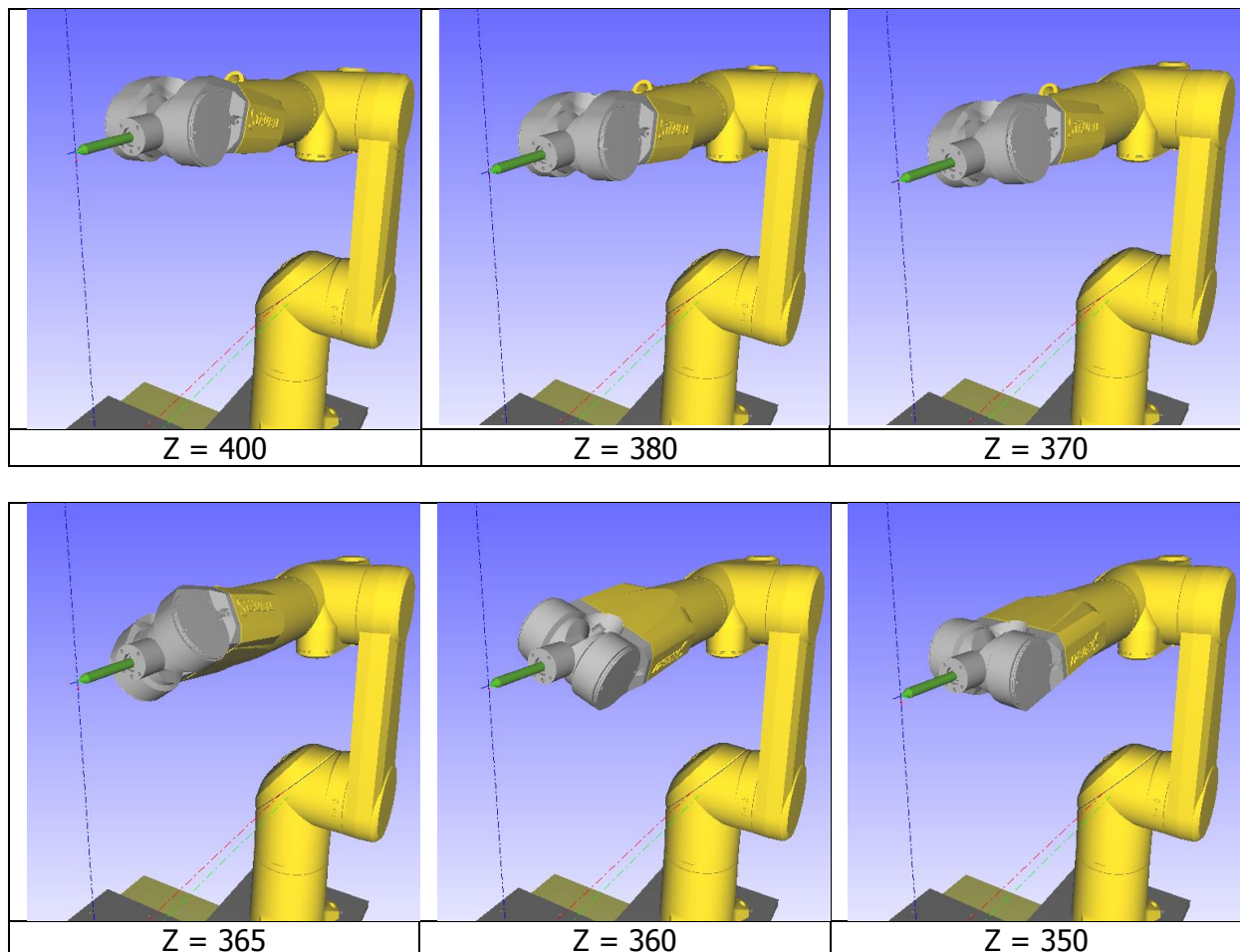
Dans le cas générale, un point cartésien donné peut être atteint selon 8 configurations différentes (2^3) par un bras anthropomorphe. Toutefois, en fonction des différentes limites de chaque articulation, il se peut que plusieurs de ces configurations ne soit pas atteignables pour une position cartésienne donnée.



6.6 Singularités

Une singularité est un point particulier à la limite entre 2 configurations physiques du bras du robot (combinaison *lefty-righty*, *above-below*, *flip-noFlip*). Lorsque le robot est proche d'un point de singularité, un déplacement cartésien faible engendre un mouvement articulaire important, ce qui peut provoquer un déclenchement de la puissance du robot en raison d'une vitesse d'axe trop élevée. Certains contrôleurs sont plus aptes que d'autres à gérer ces passages, et réduisent la vitesse de déplacement linéaire à l'approche de ces points.

L'exemple ci-dessous montre le passage d'un point de singularité *flip – no-flip*, où les axes 4 et 6 du robot anthropomorphe tendent à devenir parallèles au cours d'un mouvement cartésien linéaire. En une distance très courte (5 mm: $360 < Z < 365$), ces 2 axes du robot effectuent une rotation de près de 180° . La position de départ est (TX60L) : (650, 18, 400, 0, 90, 0).

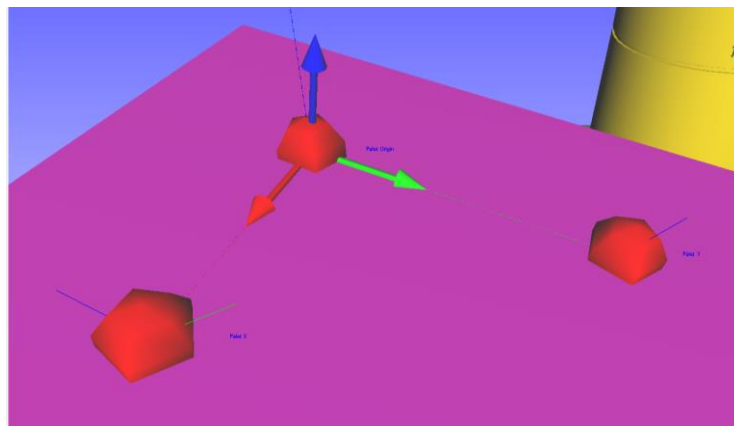


6.7 Apprentissage d'une position orientée avec le robot

Dans la plus part des cas, l'apprentissage d'une position de référence cartésienne ne peut pas être réalisée par le biais d'une position unique. L'orientation d'une position est généralement basée sur une méthode par calcul, sur la base de plusieurs positions apprises. Une position orientée est appelée *frame*.

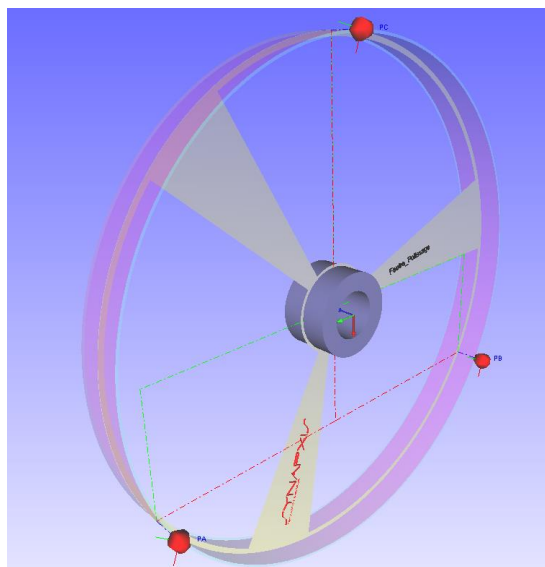
6.7.1 frame plan

Le calcul de l'orientation d'un plan (palettisation, usinage, vision, etc) peut être réalisé à l'aide de 3 points appris dans le plan en question (O, X, Y). Par produit vectoriel $\vec{OX} \wedge \vec{OY}$, le vecteur directeur du plan est connu (rotation RX et RY). La rotation RZ est finalement déterminée par le vecteur entre deux points (généralement OX). La translation du frame (X, Y, Z) correspond généralement à celle du point *Origine*, mais peut également être prendre la valeur d'un autre point.



6.7.2 frame cylindrique

Dans le cas d'un frame cylindrique, il faut définir l'axe de rotation. 3 points appris sur le périmètre du cylindre permettent de calculer le centre du cercle. Ces trois points doivent appartenir à un plan dont le vecteur directeur est parallèle à l'axe de rotation (Z). Ainsi, la position de l'axe est connue grâce à la position du centre (X, Y, Z), ainsi que 2 rotations grâce au vecteur normal de ce plan (RX, RY). RZ est fixé de manière arbitraire.



7 Mouvements programmés avec Synapxis

7.1 Mouvement articulaire

Le système de coordonnées articulaire est utilisé pour positionner le robot dans les zones de travail, et impose la *configuration* du bras. Il est important d'utiliser des positions articulaires pour lesquels il n'y a pas d'*ambiguïté* au niveau de la configuration avant de réaliser un mouvement cartésien.

Un mouvement vers une position articulaire est exécuté de manière à ce que chaque axe effectue une variation de 0 à 100% de son mouvement simultanément. Ce type d'interpolation est appelé interpolation *joint*, et correspond au mouvement le plus naturel du robot. Il est d'autre part généralement le plus rapide.

Les instructions *robot* relatives à ce système de coordonnées sont :

movej	mouvement vers une position articulaire
herje	retourne la position articulaire courante du robot

Les autres instructions relatives aux positions articulaires permettent d'accéder aux valeurs des différents axes d'une position articulaire (lecture et écriture) :

jointSetValue	écriture de la valeur de d'une coordonnée articulaire
jointValue	lecture de la valeur d'une coordonnée articulaire

7.2 Mouvement cartésien

7.2.1 Tool

Avant d'exécuter un mouvement vers une position cartésienne, ou d'apprendre une position cartésienne, il est primordial de définir le *tool* courant du robot. Le robot ne peut considérer qu'un seul *tool* à la fois, par contre une application peut utiliser plusieurs *tools* différents (tool d'apprentissage, de travail, d'usinage, etc...).

setTool	modifie le tool courant du robot
---------	----------------------------------

7.2.2 Mouvement cartésien - interpolation joint

Par défaut, un mouvement vers une position cartésienne est effectué selon une interpolation *joint*. Il est possible de spécifier une position d'*approche*, qui est soustraite selon l'axe -Z de la position considérée (multiplication matricielle). La **configuration** du bras n'est jamais altérée par un mouvement cartésien.

move	exécute un mouvement avec une interpolation <i>joint</i> vers une position cartésienne, approchée selon -Z d'une distance d'approche.
------	---

7.2.3 Mouvement linéaire

Un mouvement vers une position cartésienne peut être réalisé de manière rectiligne (interpolation linéaire, *straight*).

moves	exécute un mouvement avec une interpolation linéaire vers une position cartésienne, approchée selon -Z d'une distance d'approche.
-------	---

7.2.4 Mouvement circulaire

Un mouvement circulaire peut être réalisé en spécifiant une position cartésienne intermédiaire en plus de la position cartésienne finale.

movec	exécute un mouvement avec une interpolation circulaire vers une position cartésienne, en passant par un point intermédiaire permettant de définir l'arc de cercle.
-------	--

7.2.5 Frames

Un *frame* est une position cartésienne (6 coordonnées x, y, z, rx, ry, rz) calculée à partir de 3 positions cartésiennes. Ces 3 positions (*Origine, ptX, ptY*) définissent un plan, dont le vecteur directeur (ou normale) correspond au produit vectoriel $OX \wedge OY$. Les coordonnées x, y, z coïncident avec celle de la position *Origine*. Les rotations rx et ry sont fixées par le vecteur directeur du plan (il correspond le vecteur vz de la matrice de la position). Finalement, la rotation rz est donnée par le vecteur OX (parallèle au vecteur vx de la matrice; vy est le obtenu par produit vectoriel: $vy = vz \wedge vx$)

Si nécessaire, une 4^{ème} position cartésienne peut être spécifiée si la position du *frame* (x, y, z) est différente de la position *Origine* utilisée pour calculer l'orientation (le *frame* peut au final ne pas être dans le plan utilisé pour calculer son orientation).

frameCompose	calcul une position cartésienne orientée
distanceTo	calcul la distance entre 2 positions cartésiennes
deltaTo	calcul la différence entre 2 positions cartésiennes
alignTo	calcul une position cartésienne alignée (Z) à l'axe du système de référence le plus proche

7.3 Vitesses et accélérations

La vitesse d'exécution d'un mouvement est donnée par la multiplication des vitesses *monitor* et *program*.

7.3.1 Vitesse moniteur

La vitesse *moniteur* est imposée par le MCP du robot. Même si elle peut être changée (0.05% - 100%) de manière asynchrone pendant l'exécution d'un programme, elle doit être considérée comme constante. Elle est variée uniquement pendant les phases de test et de débogage d'un programme.

Elle doit être réglée à 100% pour le cycle normal du robot. De cette manière, il est possible d'aller plus lentement en la diminuant, mais jamais plus rapidement.

C'est également la *vitesse moniteur* qui est considérée pour les mouvements effectués en mode manuel au MCP.

7.3.2 Vitesse programme

La *vitesse programme* est modifiée, au sein d'un programme, à l'aide d'une instruction, permettant de modifier la vitesse d'exécution des mouvements suivants (articulaires et cartésiens). C'est avec la *vitesse programme* qu'une séquence de mouvement peut être réalisée lentement par rapport à une séquence plus rapide. Elle peut prendre une valeur comprise entre 1-100%. La vitesse maximale du robot est obtenue lorsque les vitesses *moniteur* et *programme* sont à 100%.

setSpeed	écriture de la valeur de la vitesse programme courante
speed	lecture de la valeur de la vitesse programme courante

7.3.3 Vitesse linéaire

Lorsqu'un mouvement cartésien est effectué selon une interpolation linéaire, c'est la vitesse *linéaire* qui est considérée, en mm/s. Celle-ci est respectée pour une vitesse *moniteur* égale à 100%.

setSpeedLinear	écriture de la valeur de la vitesse linéaire courante
----------------	---

7.3.4 Accélération et décélération

Les changements de vitesse se font en fonction des facteurs d'accélération et décélérations courants (début d'un mouvement, fin d'un mouvement, transition entre deux mouvements à vitesse différentes). Ces deux facteurs peuvent prendre une valeur comprise entre 1-100%. Il faut veiller, lors d'une accélération de 100%, que la masse embarquée sur le robot ne dépasse pas sa charge *nominale*, pour laquelle les capacités mécaniques du robot sont dimensionnées.

setAccel	écriture de la valeur de la vitesse linéaire courante
----------	---

7.4 Enchaînement de mouvements

Par défaut, les mouvements sont exécutés de manière synchrone avec l'exécution du programme, et chaque mouvement est exécuté jusqu'à l'arrêt complet du robot avant l'enchaînement avec le mouvement suivant. Il est toutefois possible de réaliser une suite de mouvement sans que le robot ne marque d'arrêt au point, ou qu'il ne fasse qu'approcher le point sans y passer. Il est important de bien différencier les notions qui entrent en considération dans l'enchaînement des mouvements.

7.4.1 Synchronisation

Lors de l'envoi d'une commande au robot (lecture : position courante, tool courant, vitesse programme courante, etc... ; écriture : setTool, setSpeed, setAccel, etc...), le système attend toujours, avant de continuer, une réponse de confirmation de la part du robot, confirmant que la commande a été bien prise en compte. Contrairement aux autres instructions, les instructions de mouvement provoquent deux événements décalés dans le temps :

- Le mouvement commandé au robot est ajouté dans sa *pile de mouvement*. Une fois le mouvement validé et considéré, une première confirmation est envoyée par le robot. L'exécution du mouvement par le robot commence à se faire alors que la réponse à cette commande est déjà envoyée.
- Via une 2^{ème} requête, il est possible de demander au robot qu'il envoie une confirmation de *fin de mouvement* une fois que le robot a terminé d'exécuter les mouvements commandé

(*pile de mouvement* vidée). C'est elle qui crée l'attente du programme sur la fin de l'exécution du mouvement.

Dans le cas d'un fonctionnement en mode *synchronisé*, une commande de confirmation de fin mouvement est toujours envoyée de paire avec chaque commande de mouvement. La pile de mouvement ne **comporte jamais donc plus de un seul mouvement**.

Dans le cas d'un fonctionnement en mode *désynchronisé*, la *pile de mouvement* peut se remplir de plusieurs commandes de mouvement. Il est possible de resynchroniser le programme après plusieurs mouvements, pour par exemple vérifier l'état d'un capteur (détection de pièce) ou l'activation du préhenseur (serrage de la pièce dans la palette).

setSynchronizedMove	active/désactive le mode synchronisé
waitEndMove	permet de resynchroniser le programme avec l'exécution de la pile de mouvements

7.4.2 Arrêt au point

Lors d'un mouvement AB de A vers B, le robot démarre à l'arrêt depuis le point A, se déplace vers B. Deux cas de figure sont alors possibles :

- Il marque un arrêt et se stabilise au point B. ($v_{\text{linéaire}} = 0$ mm/s). C'est le cas général et par défaut, et est lié au paramètre *break* des instructions de mouvement, qui par défaut vaut *true*. L'arrêt au point est imposé en mode *synchronisé*.
- Pour qu'il n'y ait pas d'arrêt au point B (*break* = *false*), il faut que le mouvement suivant BC soit déjà connu avant qu'il n'arrive au point B. Pour des considérations logiques de cinématique (accélération/décélération), une anticipation est donc nécessaire, et c'est donc uniquement en mode désynchronisé qu'il est possible et pertinent de demander un arrêt au point ou non.

movej, move, moves, movec	instructions d'exécutions de mouvement, avec le paramètre <i>break</i> à <i>true</i> ou <i>false</i> en fonction du comportement désiré
------------------------------	---

7.4.3 Passage au point

Lorsqu'un mouvement ABC est commandé au robot sans arrêt au point B (*break* = *false* et mode *désynchronisé*), le robot effectue un « lissage » au point B, selon les paramètres de *blending* courants.

- si les paramètres de blending sont non-nuls (distances *leave* et *reach*, qui décrivent 2 sphères autour du point), le robot approche le point B jusqu'à la distance *reach*, puis enchaîne avec un mouvement de transition, proche de B et en direction de C, jusqu'à atteindre la distance *leave* par rapport à B. Le mouvement vers C est ensuite réalisé.
- si les paramètres de blending sont nuls (ou très petits), le robot passe obligatoirement par le point B, sans s'y arrêter et s'y stabiliser. En fonction de l'angle compris entre les segments AB et BC, le robot ralentit plus ou moins, afin de garantir le passage par B, mais en enchaînant dès que possible avec le mouvement BC.

setBlending	écriture des paramètres de blending
-------------	-------------------------------------

8 Programmation

L'exécution d'un programme par un système informatique est intimement liée au fonctionnement de celui-ci. Le présent chapitre expose les rudiments du fonctionnement d'un tel système, et celui d'après les bases de la programmation.

8.1 Système informatique : les rudiments de son fonctionnement

Quelques définitions :

- Carte mère : circuit imprimé sur lequel se trouvent les principaux composants du système informatique.
- CPU : *Central Processing Unit*, le processeur principal de la carte mère.
- GPU : *Graphics Processing Unit*, le processeur dédié aux calculs d'affichage. Ils ont une capacité de calcul extrêmement élevé (parfois plus élevée que celle du CPU). Certains programmes exploitent cette puissance de calcul en déléguant au GPU une partie des opérations (*OpenCL*, *CUDA*). Il peut être intégré sur la carte mère, ou additionnel (carte graphique).
- Bus : électronique de la carte-mère qui permet de faire transiter les informations entre les différents composants: CPU <-> RAM, etc.
- Système 32bits : la carte mère est basée sur un bus de données et un bus d'adresse de 32 bits, capable d'adresser $2^{32} = 4'294'967'296$ octets différents (d'où la limitation à 4Go de mémoire vive pour un tel système, famille des processeurs *Intel X86* par exemple).
- Système 64bits : l'adressage de la mémoire et les données sont basés sur 64 bits, ce qui augmente considérablement la capacité de mémoire, ainsi que l'étendue des nombres représentés nativement (une valeur 64 bits peut cependant très bien exister sur un système 32 bits, de manière non-native).

8.1.1 Cœur

Le cœur (*core*) est le centre du système informatique, où se situe le système électronique qui exécute les instructions du programme, écrites en langage *assembleur* (langage du plus bas niveau). En fonction de la technologie, un cœur peut exécuter un ou plusieurs *threads* à chaque cycle d'horloge.

8.1.2 Processeur

Le processeur est le circuit intégré (puce, ou *chip*) qui contient le ou les cœurs. Les premiers processeurs étaient mono-cœur (*Pentium I, II*). Maintenant que la fréquence d'horloge (cadencement du processeur) a atteint son maximum, l'augmentation de la puissance d'un processeur se traduit par l'augmentation de son nombre de cœur. Un des premiers processeurs multi-cœur était le *Pentium D* (2 cœurs), et les processeurs actuels (2014) possèdent entre 2 et 6 cœurs (Intel Core i7). Les *superordinateurs* multiplient la puissance de calcul en augmentant le nombre de processeurs (jusqu'à plusieurs centaines de milliers!).

8.1.3 Multitâches

Le fonctionnement *multi-tâches* d'un système informatique est assuré par le *séquençement* (ou *ordonnancement*) des différents *threads*: chaque *thread* attribué au cœur est exécuté pendant une durée déterminée (par le système d'exploitation), avant d'être *suspendu* et de laisser la place au thread suivant. La puissance de calcul et la vitesse de cadencement rend d'un point de vue utilisateur le système parfaitement multitâche, mais des *threads* ne peuvent être réellement exécutées en parallèle que depuis l'apparition des systèmes multi-cœurs.

8.1.4 Processus

Un processus est un ensemble de *threads* nécessaires au fonctionnement d'une application. Par exemple, *Mircosoft Word*, une "simple" application de traitement de texte, nécessite pour son fonctionnement plusieurs *threads* (~15 !!) : gestion de l'affichage, saisie au clavier, check-orthographe et syntaxe, enregistrement et accès fichiers, surveillance de la licence, etc. Chaque *processus* utilise un espace mémoire qui lui est réservé.

8.1.5 Thread

Un *thread* (ou *tâche*, *file d'exécution*) représente l'exécution d'une suite d'instructions par un cœur. Il est exécuté à la cadence du processeur, avec une *priorité* qui lui est propre (*basse, élevée, critique, etc*).

Un *thread* est créé et attribué à une *routine principale*. Une fois que l'exécution de celle-ci est terminée, le *thread* s'arrête et est normalement détruit. La routine principale peut être une boucle qui se répète jusqu'à ce que le *processus* auquel il appartient soit terminé (mise à jour d'une fenêtre graphique, surveillance d'un capteur, dialogue avec un périphérique, etc).

Tous les *threads* d'un processus ont accès au même espace mémoire, celui du *processus* auquel ils appartiennent.

Si un *thread* est séquencé et qu'il n'a temporairement plus besoin de ressource (plus de traitement à réaliser), il est possible de redonner la main au processeur, qui séquencera alors directement le thread suivant. Ainsi, il est possible d'avoir un grand nombre de *processus* actifs, chacun avec un grand nombre de *threads*, sans que le processeur soit surchargé (ce qui est le cas sous Windows dans ~97% du temps).

8.1.6 Mémoire

La mémoire *vive* d'un système informatique (*RAM, Random Access Memory*) assure le stockage temporaire des données pendant le fonctionnement du système, mais ne permet pas de stockage permanent. C'est le disque dur et les disques réseaux/amovibles qui assurent ce rôle.

La mémoire préférentiellement utilisée par le processeur est la mémoire *cash* (*level 1, 2*), qui fonctionne très rapidement mais dont la taille est limitée. Cette mémoire *cash* (~1Mo) assure le tampon entre la mémoire *principale* (~1-16Go, *level 3*). Pour l'utilisateur (le programmeur d'application), la gestion du fonctionnement de ces différentes mémoires *vives* est transparente, elle est gérée par le processeur, la carte mère et le compilateur du programme.

Il est toutefois important de distinguer 2 types d'utilisations de cette mémoire, qui cohabitent lors de l'exécution d'un programme : la mémoire *de masse*, et la mémoire *stack*.

8.1.6.1 Mémoire de masse (heap)

La mémoire de masse est la mémoire dans laquelle sont notamment chargés :

- Le code exécutable de l'application: une application peut être lancée plusieurs fois (explorateur de fichier, calculatrice, etc). Son code exécutable est toutefois chargé une seule fois en mémoire, et utilisé par plusieurs processus simultanément. Cette zone mémoire n'évolue normalement pas pendant l'exécution du programme. Certains logiciels peuvent modifier l'organisation du code exécutable au cours de son exécution, de manière à pouvoir biaiser certains mécanisme de surveillance (virus) ou pour leur propre protection.
- Les données nécessaires au fonctionnement de l'application : un processus de lecture mp3 va charger les données du fichier audio dans une partie de la mémoire de masse avant d'y accéder : calcul, traitement, etc.

8.1.6.2 Mémoire stack (pile d'appel)

La mémoire *stack* est l'espace mémoire dédié à un *thread*, lui permettant de "vivre" et de réaliser les fonctions de l'application:

- Lors de l'appel d'une autre routine (*sous-routine*, *routine appelée*) depuis une routine donnée (*routine appelante*), le programme exécute une portion de code exécutable située à un autre endroit dans la mémoire de masse. Cela est appelé un *saut*. Avant de réaliser ce saut, il est nécessaire de mémoriser la position actuelle dans le code exécutable afin de pouvoir y revenir une fois la *routine appelée* terminée.
- Si la routine appelée comporte des paramètres, la routine *appelante* doit lui mettre à disposition ces variables (par *valeur* ou par *référence*), également via cette mémoire *stack*.
- Les variables locales d'une routine sont créées sur la stack. De cette manière, si une routine est appelée par plusieurs thread, ou qu'elle se réappelle elle-même (récursivité), chaque *occurrence d'exécution* comporte des variables locale qui lui sont propre, ce qui est absolument nécessaire pour le fonctionnement d'une application.
- Lorsque la fonction appelée (sous-routine) se termine, si celle-ci a un type de retour non-nul (c'est alors une fonction!), la valeur retournée est transmise à la fonction appelante via cette mémoire stack.

Cette mémoire *stack* est utilisée par le cœur comme une *pile* d'assiette (d'où son nom) :

- Chaque assiette représente une routine.
- La première assiette est le *point d'entrée* du *thread*. Il s'agit de la routine principale (fonction *main* en C, C++, Java).
- Lors de l'appel d'une sous-routine (*saut*), une assiette est ajoutée:
 - *valeur retournée*
 - *paramètres d'entrées*
 - *variables locales*
 - *position courante avant le saut*
 - *état des registres (variables spécifiques internes au cœur du processeur)*
- Lorsqu'une sous-routine se termine, la valeur de retour (si type non-nul) est écrite dans l'espace qui lui a été réservé sur la stack. Lorsque le traitement de la sous-routine appelante reprend, la valeur de retour est utilisée et l'assiette est enlevée.
- L'appel de sous-routine peut se cascader dans une très grande mesure (plusieurs centaines de niveaux !), mais a toutefois une limite qui correspond à la taille de espace mémoire réservé à la stack. Un dépassement de cette capacité provoque un *stack-overflow*, donc un crash de l'application.
- Quand la première routine appelée se termine, la première assiette est enlevée. La table est rase, et le thread s'arrête.
- La variable qui mémorise quel est le niveau de la pile couramment en cours d'exécution s'appelle le *stack-pointer*.
- Lors du séquençement, le processeur "abandonne" l'exécution du *thread* courant (exécution suspendue, la pile n'évolue plus), et reprend le traitement du thread suivant, avec sa propre pile. Chaque *thread* attribué à un *cœur* est ainsi traité cycliquement, et sa *stack* permet d'en mémoriser l'état.
- Une *interruption* permet d'exécuter un thread de manière asynchrone (*IRQ*). Il est possible de demander au système de déclencher l'exécution d'un thread lorsqu'un *signal* spécifique est activé (clavier, souris, port sériel ou parallèle, etc). Le processeur interrompt alors le cycle de séquençement des threads enregistrés et exécute le code lié à l'interruption dans un autre *thread*, avant de reprendre le cycle normal.

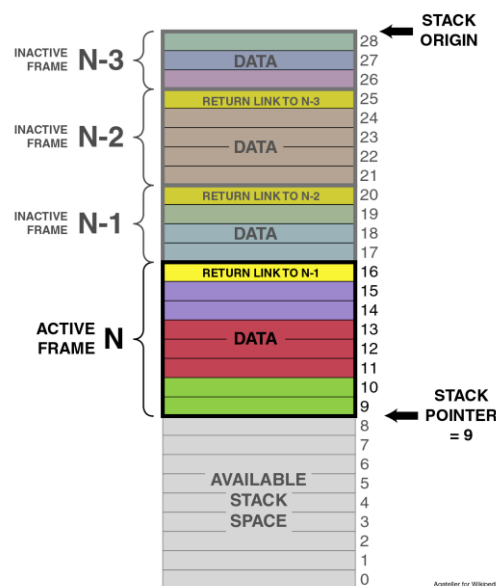


Figure 6 Représentation d'une pile d'appel (stack)

8.2 Rappels des notions fondamentales de programmation

Dans le cadre de ce cours, la programmation des exercices est effectuée à l'aide du logiciel *SYNAPXIS*. Ce logiciel, développement de la HE-ARC/UR-LPR, est destiné au pilotage de machine de production robotisées, notamment pour des opérations d'usinage et de suivi de trajectoire, handling, assemblage, etc. Il permet le pilotage de un ou plusieurs robots, de marques *Adept*, *Kuka*, *Stäubli*, et *Mitsubishi*.

Le langage de programmation de *Synaxis* est un langage *procédural*, multitâches. Dans la suite de ce document, les propriétés supportées par ce langage sont identifiées par (✓), celles qui ne le sont pas par (✗).

8.2.1 Type de programmation

Il existe 2 grands types (paradigme) de programmation répandus :

- programmation *procédurale* : le code est composé de *procédures*, appelées entre elles. C'est le cas notamment du *C*, *Basic*, ainsi que du langage de programmation de Synaxis (✓). Ce paradigme est également appelé *Impératif*.
- programmation *objet* : le programme est composé d'*objets*. Chaque type d'objet définit des *méthodes* (code) et une structure de données (membres) qui lui sont propres. Les membres d'un objet peuvent être d'autres objets. L'appel d'une *méthode* passe donc par un objet existant (instancié). C'est le cas du *C++*, *Java*, *C#*, etc (✗).

8.2.2 Programme

- Le mot "*programme*" désigne l'ensemble du code nécessaire à la réalisation d'une application (*software*, *logiciel*).
- Un programme est composé de un ou plusieurs *modules* (ou *librairies*), permettant d'organiser (regrouper) le code.
- Un module est composé de *procédures*.

8.2.3 Procédures

Une procédure (ou *routine*, *sous-routine*) est un ensemble de ligne de code, destiné à exécuter une fonctionnalité donnée : calcul, communication, affichage, etc.

- Une procédure possède un nom unique (✓).
- En fonction des langages, une procédure peut être appelée par plusieurs tâches simultanément (✓).
- En fonction des langages, une procédure peut s'appeler elle-même (*récurtivité*) (✓).
- En fonction des langages, une procédure est appelée à l'aide de l'instruction *call* (✗):
 - `call MyProc()`
- Dans d'autres langages, seul le nom de la procédure est spécifié pour l'appel (✓):
 - `MyProc()`
- Une procédure peut définir des *paramètres* (✓).
 - `MyProc(vMin, vMax, v)`

8.2.4 Paramètres

Les paramètres passés à une fonction sont définis par leur ordre, leur type et leur nom.

- Le nom d'un paramètre doit être unique, parmi les autres paramètres ainsi que les *variables locales* de la procédure.
- Le nom du paramètre est *local*, c'est-à-dire qu'il est considéré uniquement à l'intérieur de la procédure. Dans le code où est fait l'appel à cette procédure, la variable passée comme paramètre peut très bien être une constante, une variable *locale* ou *globale*, un élément d'un tableau, ou même un paramètre de cette fonction.
 - `MyProc()` // *pas de paramètre d'entrée*
 - `MyProc(2, "hello")` // *constantes*
 - `MyProc(MyProc2())` // *imbrication de fonction*
 - `MyProc(param1, param2)` // *"propagation" des paramètres*
 - `MyProc(a+b, c)` // *variables locales, opérations*
 - ...
- Certains langage permettent d'appeler une procédure sans que tous les paramètres ne soient spécifiés (usage de *valeurs par défaut*) (✗).
- Un paramètre peut être passé par *valeur* ou par *référence*. Certains langage impose un mode ou l'autre, d'autres permettent de choisir entre l'un ou l'autre de ces modes.

Le passage de paramètre doit être privilégié par rapport à l'usage de variables globales, de manière à obtenir un code qui soit le plus réutilisable et fiable possible.

8.2.5 Fonction et type de retour

Une *fonction* est une procédure qui *retourne* une variable. Ce type de fonctionnalité, qui n'est pas supporté par tous les langages, permet une écriture encore plus condensée et lisible du code (✓).

- Le type de retour de la fonction doit être défini.
- En fonction des langages, le retour peut être fait *par valeur* (✗) ou *par référence* (✓).
- Une fonction ne peut retourner qu'une seule valeur, ce qui parfois est limitatif. Il faut recourir à des paramètres passés par *référence* pour obtenir plus d'une valeur (ou alors un retour avec une variable de type *structuré*).
- Une fonction qui ne retourne pas de variable est par définition une *procédure*. Il est également possible de parler de retour de type *nul*, ou *void* (✓).

Exemple de code par passage de paramètre: un paramètre est passé à la *procédure* `getValue(value)` et celle-ci modifie sa valeur lors de son exécution.

```
getValue(v)
print(v)
```

Exemple de code avec une fonction : la *fonction* `getValue()` retourne la valeur.

```
print(getValue())
```

8.2.6 Valeur et référence

Lorsqu'une variable est passée comme paramètre à une procédure, il y a 2 cas de figure possible:

- 1) Passage par *valeur* : la valeur de la variable est copiée et c'est cette copie qui est considérée par la procédure. Ainsi, si la procédure modifie cette copie, l'état de la variable originale n'est pas modifié (✗). Le seul avantage du passage par valeur est la *sécurité* qu'une variable passée à une procédure ne puisse pas être modifiée par celle-ci.
- 2) Passage par *référence* : l'adresse de la variable est passée à la procédure (✓). Cela permet à la procédure de *lire* et d'*écrire* cette variable, donc de voir ce paramètre comme un paramètre d'*entrée* ou de *sortie*.

Remarques :

- Une référence peut être *cassée* en insérant une opération neutre: `MyProc(v+0)`. Ce n'est pas la variable `v` qui est transmise à la procédure, mais le résultat de l'addition `v+0`.
- Dans le cas du passage *par valeur*, il faut garder à l'esprit que la variable est **copiée**. S'il s'agit d'une variable de type simple (entier, réel, etc), la copie est immédiate. Par contre, si la variable est de type *composé*, par exemple un *tableau* contenant un grand nombre d'éléments, le temps de copie et l'espace mémoire nécessaire augmente en fonction de la taille de la variable. Attention aux performances !
- Ces considérations s'appliquent également à la variable retournée.

8.2.7 Types de variables

En fonction des langages, les types supportés peuvent être différents. Un langage de bas niveau, c'est-à-dire très proche du matériel (le processeur), se base sur les types les plus *primitifs*, et permet de créer des types *structurés* ainsi que des *objets*. Moins facile à prendre en main, et nécessitant plus de rigueur, ces langages permettent une optimisation des performances (manipulation de la mémoire, etc).

Par exemple, les types de variables couramment utilisés en c++ sont :

- primitifs
 - o bool
 - o char
 - o signed/unsigned short, signed/unsigned long
 - o float, double
- tableaux
 - o char * (pointeur sur une chaîne de caractère)
 - o int [], ...
- composés
 - o struct
 - o class

Un langage de plus haut niveau, c'est-à-dire moins proche du processeur, peut se baser sur des types plus évolués, en automatisant par exemple certains mécanismes complexes (allocation et restitution de la mémoire, etc). Ils sont généralement plus faciles à prendre en main et nécessitent moins de rigueur, mais sont moins efficaces en terme de performances.

Par exemple, les types variables supportés par *Synapxis* sont les types simples, des types dédiés à la robotique et des types composés. La documentation de *Synapxis* explique en détail les différentes spécificités de ces types.

- simples
 - o booléen
 - o entier
 - o réel
 - o string
- dédiés
 - o position cartésienne
 - o position articulaire
- composés
 - o tableau
 - o structure
 - o buffer

8.2.8 Variables locales

Les variables *locales*, appelées parfois variables *auto*, sont des variables utilisées pour des opérations locales, et doivent être considérées comme *éphémères*. Elles appartiennent à la procédure dans laquelle elles sont déclarées, et n'existent que pendant la durée d'exécution de la procédure. D'un point de vue *mémoire*, elles sont créées sur la *pile d'exécution (stack)* lors de l'appel de la fonction, et détruites lorsque celle-ci *retourne* (se termine).

Si une procédure est appelée depuis plusieurs *threads*, ses variables locales sont créées sur la *stacks* de chaque *thread*. De la même manière, si une procédure est appelée de manière récursive par un *thread*, ses variables locales sont créées pour chaque appel.

Il n'y a donc pas d'interférence entre les différentes occurrences d'exécution d'une procédure et ses variables locales.

Remarques :

- ***L'emploi de variables locales chaque fois que c'est possible est par définition la meilleure solution pour s'assurer qu'il n'y ait pas d'interférence entre les différentes variables d'une application*** (par opposition aux variables *globales*).
- Une erreur fréquente est par exemple de voir déclarée une variable pour l'itération d'une boucle *for* (par exemple *i*) comme une variable globale. Le risque de bug existe alors dès que cette variable est accédée depuis plusieurs procédures, ou depuis plusieurs *threads*.
- Les variables locales étant propres à une routine (*privées*), elles ne surchargent pas les autres routines lors de l'édition et de la lecture du code. Il se peut très bien qu'une routine ait besoin d'un grand nombre de variables locales.
- En opposition à cela, les variables globales sont visibles pour toutes les procédures, il est donc important d'en limiter le nombre.

8.2.9 Variables globales

Les variables globales ont la particularité d'exister en dehors de tout cycle de vie d'une procédure. Leur emplacement mémoire est situé en dehors de la *stack*, dans la mémoire de masse (*heap*). Elles sont ainsi accessibles en tout temps par toutes les procédures, indépendamment de la tâche considérée. (✓)

L'utilisation de variables globales doit être réduite au minimum, car leurs avantages peuvent également se révéler être un piège en terme de fiabilité, d'organisation et de clarté pour le code:

- Une variable globale pouvant être accédée par plusieurs tâches simultanément (*read+write*), une erreur de conception peut provoquer des dysfonctionnements sérieux du programme.
- Par opposition aux paramètres d'entrée d'une fonction qui doivent impérativement être considérés, une variable globale peut très bien être "oubliée" lors de l'écriture de la routine (pas d'initialisation, etc).

Ainsi, l'emploi d'une variable globale devrait **répondre au moins à une des 3 nécessités suivantes**:

- 1) Dialogue entre tâche (utiliser un *mécanisme de verrouillage* pour en gérer l'accès).
- 2) Maintenir des données lorsqu'aucune tâche n'est active. Ce genre de données est généralement enregistré dans un fichier, chargé au démarrage de l'application.
- 3) Maintenir l'état du processus lorsqu'aucune tâche n'est active (état, compteur, etc).

Il est important de veiller à accéder le moins possible aux variables globales déclarées. En d'autres termes, leur nom doit apparaître le moins possible dans le code:

- Encapsulation dans des procédures dédiées (accesseurs) : *SetVar(..)*, *GetVar()*.
- Passage par paramètres: une variable globale peut très bien être passée en paramètre aux différentes *procédures*. Celles-ci la traitent alors de la même manière qu'une variable *locale*.

8.2.10 Syntaxe

Chaque langage possède sa propre syntaxe, voici quelques notions générales importantes :

- Le code est composé d'*instruction* et d'*opérateur* qui accèdent aux *variables*. Un *statement* est un bloc de code composé d'un *opérateur/instruction* principal, puis d'autres éventuelles *opérateurs/instructions* secondaires :

○ // exemple	// opérateur principal
○ <code>c = 2*a+b</code>	<code>// =</code>
○ <code>MyFunc(2*a, sin(b))</code>	<code>// call (appel à une routine)</code>
○ <code>print(clock())</code>	<code>// print</code>
○ <code>return a>b ? a:b</code>	<code>// return</code>

- Chaque *statement* est séparé des autres. Certains langage utilisent un caractère particulier (point-virgule pour le C++ par exemple), et pour d'autres c'est la fin de la ligne de code qui marque cette séparation (✓) :

- `// tout sur une ligne: possible en C++, mais très peu lisible:`
`if (a > b) { print("a > b"); } else { print("b >= a"); }`

- `// le même code sur plusieurs lignes (✓)`
`if(a > b)`
 `print("a > b")`
`else`
 `print("b >= a")`
`end`

- Les éléments de *structure de contrôle* permettent de régir le fonctionnement du code au travers des tests et des boucles (*branching* et *looping*)
 - test : *if-else-end*, *switch-case-end*
 - boucles : *do-until*, *while-end*, *for-end*, *repeat-end*
- L'*indentation* du code en fonction de son niveau dans la *structure de contrôle* permet d'augmenter la lisibilité du code. Elle doit impérativement être respectée pour faciliter la relecture du code, même si généralement le code peut être exécuté sans qu'elle soit strictement respectée.

- `// le même code sans indentation est moins lisible`
`if(a > b)`
`print("a > b")`
`else`
`print("b >= a")`
`end`

8.3 Programmation à l'aide de Synapxis

8.3.1 Dossier et Fichiers

Synapxis démarre en chargeant les différentes données présentes dans le *Répertoire de travail* courant : données 3D, configuration des robots, entrées sorties, frames, tool, outils, etc. Le dossier `\Programs` (et ses éventuels sous-dossiers) contient les fichiers des différents modules de l'application. Chaque fichier correspond à un *module*, pouvant être de deux types différents :

- a) Module *programme*, qui peut contenir des *procédures* (ou *fonctions*) et des variables globales.
- b) Module *data*, pouvant contenir uniquement des variables globales.

Chaque *module* comporte un nom unique et est enregistré dans son propre fichier. L'extension est **.mip* pour les modules *programme*, et **.dip* pour les modules *data*.

8.3.2 Procédure principale

Comme expliqué plus haut, l'exécution d'un thread (*tâche*) nécessite une procédure principale, qui est le point d'entrée de l'application. Synapxis impose une seule règle pour qu'une procédure puisse être exécutée comme point d'entrée : elle ne doit comporter aucun paramètre.

Ainsi, en théorie, n'importe quel programme n'ayant pas de paramètre peut être exécuté directement. Cette grande liberté peut toutefois être un piège en termes d'organisation de l'application.

Pour chaque projet, il est conseillé de ne définir qu'une seule et unique routine comme point d'entrée, dont le nom devrait comporter *main*. De cette manière, les différentes initialisations nécessaires au lancement d'un processus ne sont définies qu'une seule fois pour toute, même si l'application peut réaliser un grand nombre d'opérations distinctes. Aussi, la compréhension du programme lors de la lecture est facilitée si le point d'entrée est unique et facilement identifiable.

8.3.2.1 Lancement par la fenêtre Floating

Synaxis permet de lier un bouton de la fenêtre *floating* à une procédure (F5 pour afficher cette fenêtre). L'action du bouton correspondant déclenche alors l'exécution du programme sélectionné. Ces réglages se font via le menu de la fenêtre principale de Synaxis : *Option\Floating*.

Routine de lancement : elle est exécutée lors de l'action sur le bouton. Celle-ci doit être exécutée sans attente, et retourner directement. L'exécution de la routine principale doit donc être lancée dans un nouveau *thread* (tâche) à l'aide de l'instruction *taskExecute* :

```
// * * * * *
// Module:      Ex05
// Description: Lancement de la routine principale
// * * * * *

taskExecute Ex05_Main(), "mainTask"
return
```

Dans cet exemple, la routine *Ex05_Launch()* lance l'exécution de la routine *Ex05_Main()* dans la tâche "mainTask".

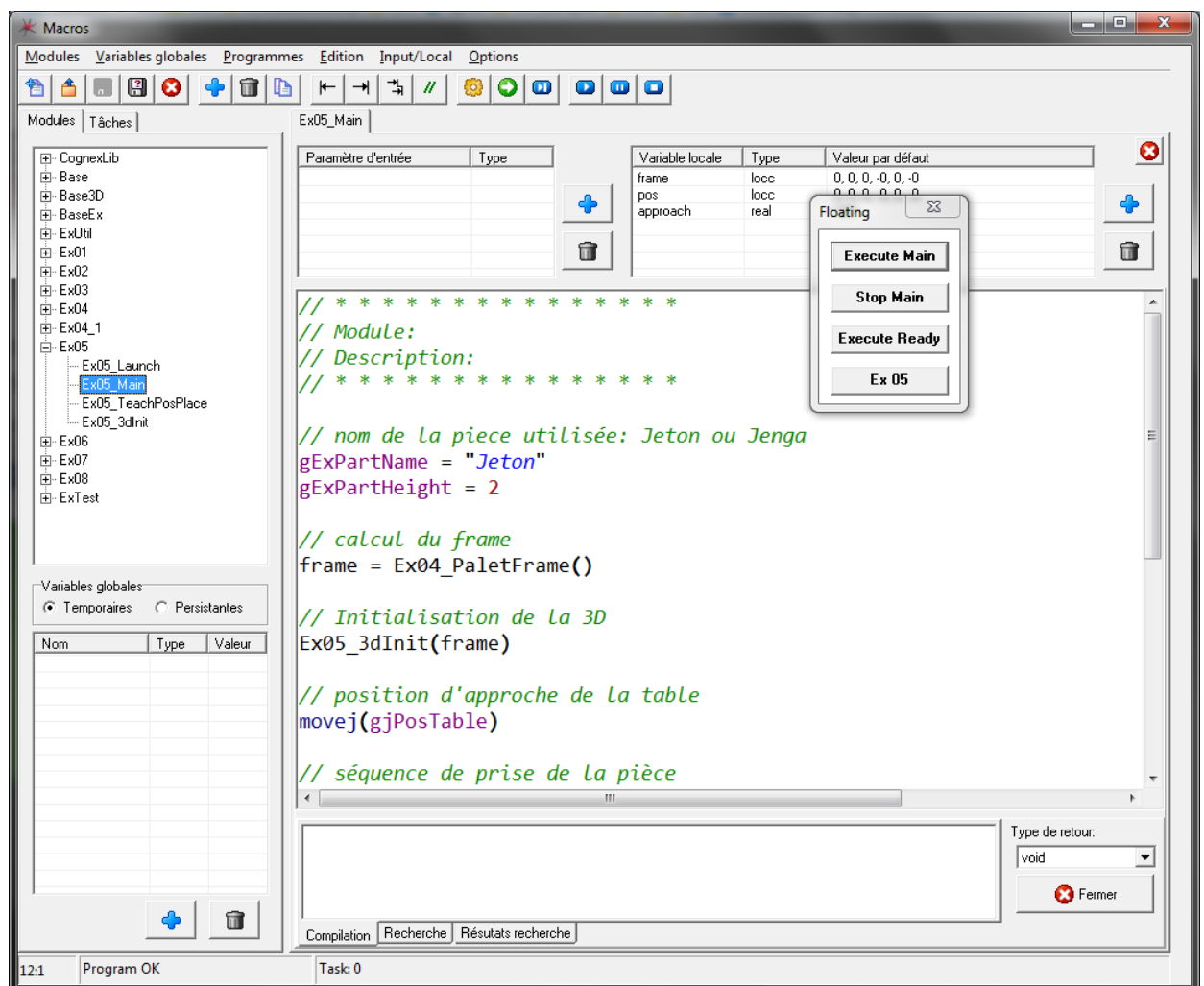


Figure 7 La routine principale

8.3.3 Variables globales : temporaire ou persistante ?

Synapxis offre la possibilité de définir une variable globale comme étant *temporaire* ou *persistante*. La différence est faite lors de l'enregistrement de ces variables dans le fichier du module correspondant :

- Une *variable globale temporaire* est enregistrée dans le fichier sans sa valeur courante au moment de l'enregistrement. De cette manière, lorsque le fichier est rechargé (par exemple redémarrage de Synapxis), la variable prend une valeur nulle (0, string vide, tableau vide, etc). La variable est donc simplement déclarée et initialisée à zéro. Ce cas de figure est utile pour les variables de fonctionnement (qui sont initialisées par le programme lui-même), des variables de mesure, calculs statistiques, etc.
- Une *variable globale persistante* est enregistrée dans le fichier avec sa valeur courante au moment de l'enregistrement. Lors du rechargement du module, la variable est déclarée et retrouve sa valeur enregistrée. Ce cas de figure est utile pour maintenir des données (paramètres de réglages, positions cartésienne ou articulaire, etc).

L'utilisation de *variables globales persistantes* ne doit être faite que lorsque cela est nécessaire. Par exemple, un tableau contenant un grand nombre de mesures obtenues au cours d'un processus prendrait un espace fichier considérable, alors que peut être seule une valeur résultante unique présente un intérêt et devrait être sauvegardée. Aussi, des valeurs enregistrées "de manière involontaires" peuvent également être la source de bug et de manque de clarté.

Toutes les informations sur le langage de programmation de Synapxis figurent dans le manuel de référence correspondant.

8.4 Charte syntaxique

Dans un but de clarté et de facilité de relecture, une *charte syntaxique* et organisationnelle est imposée pour la programmation des différents exercices avec Synapxis :

- Chaque exercice correspond à un module
- Le nom des différents modules est uniforme :
Ex01, Ex02, Ex03, ... ou *Exercice_01, Exercice_02, ...*
- Le nom des différentes procédures nécessaires à la réalisation de l'exercice (application) commence par un préfixe identique :
Ex01_Launch(), Ex01_Main(), Ex01_Init(), Ex02_PaLetGet(),...
- Le nom des variables globales portent un identifiant (g) et le même préfixe que les routines du module :
gEx01_Frame_X, gEx01_CameraName, ...
- Des procédures d'un module peuvent être utilisées dans un autre module. Les variables globales d'un autre module doivent être accédées via une routine dédiée (*accesseurs*) :
frame = Ex03_TableFrame() // variables globales dans cette routine

9 Réseaux, bus de terrain, IOs

L'interconnexion de plusieurs appareils est courante dans le monde de la technique, et il existe un grand nombre de technologies disponibles.

Les *bus de terrains*, généralement liés à un automate programmable industriel (API), sont notamment utilisés pour relier les *capteurs* et *actionneurs* d'une machine ou d'une installation. Les principales caractéristiques de ces *bus* sont la fiabilité, la robustesse face aux perturbations externe et le déterminisme (temps-réel). Ce type de liaison est généralement réservé au matériel industriel :

- CanOpen
- MODBUS
- Profibus
- EtherCAT (basé sur le matériel TCP/IP)
- ASi

Il existe également d'autres moyens pour connecter un appareil à un système informatisé (PC, API), là où par exemple le déterminisme n'est pas nécessaire. Ce type de liaison est généralement destiné au matériel tout public comme industriel :

- RS232
- RS485
- USB
- TCP/IP

9.1 TCP/IP

Un nombre de plus en plus important de périphériques sont interfaçables par connexion TCP/IP. Il est important pour l'ingénieur de comprendre les bases du fonctionnement du réseau TCP/IP, ne serait-ce parce que son propre ordinateur l'utilise se connecter au réseau et à internet.

9.1.1 Internet

Internet est un système d'interconnexion de machines qui constitue un réseau informatique mondial, utilisant un ensemble standardisé de protocoles de transfert de données. C'est un réseau de réseaux, sans centre névralgique, composé de millions de réseaux aussi bien publics que privés, universitaires, commerciaux et gouvernementaux. Internet transporte un large spectre d'information et permet l'élaboration d'applications et de services variés comme le courrier électronique, la messagerie instantanée et le World Wide Web.

Internet ayant été popularisé par l'apparition du World Wide Web, les deux sont parfois confondus par le public non averti. Le World Wide Web n'est pourtant que l'une des applications d'Internet.

9.1.2 Réseau local, Ethernet, Wi-Fi

Un réseau local, souvent désigné par l'acronyme anglais LAN de Local Area Network, est un réseau informatique tel que les terminaux y participant (ordinateurs, etc.) s'envoient des trames de données au niveau de la couche de liaison sans utiliser d'accès à *internet*. On définit aussi le LAN par le domaine de diffusion, c'est-à-dire l'ensemble des stations qui reçoivent une même trame de diffusion. Au niveau de l'adressage IP, un réseau local correspond généralement à un sous-réseau IP (même préfixe d'adresse IP). On interconnecte les réseaux locaux au moyen de routeurs.

Un réseau local sans fil est un réseau informatique qui connecte différents postes ou systèmes entre eux par ondes radio (WLAN). Le Wi-Fi est un type de réseau WLAN.

Ethernet est un protocole de *réseau local* à commutation de paquets. Toutes les machines du réseau Ethernet sont connectées à une même ligne de communication, constituée de câbles.

9.1.3 Serveur

Un serveur informatique est un dispositif informatique matériel ou logiciel qui offre des services à différents clients. Un serveur fonctionne en permanence, répondant automatiquement à des requêtes provenant d'autres dispositifs informatiques (les *clients*), selon le principe dit client-serveur. Le format des requêtes et des résultats est normalisé, se conforme à des protocoles réseaux et chaque service peut être exploité par tout client qui met en œuvre le protocole propre à ce service.

Dans le monde de la technique, un *serveur* correspond à un appareil/équipement/logiciel qui permet de réaliser une tâche spécifique (service) :

- Caméra (capteur uniquement): acquisition d'image.
- Caméra autonome (ou *intelligente*) : acquisition, traitement, analyse, etc. C'est le cas des camera *Cognex InSight* utilisées dans le cadre de ce cours.
- Système robotique : comme c'est le cas avec Synapxis, le robot est vu comme un "serveur de mouvement", auquel le logicielle se connecte pour gérer ses différents états et fonctionnalités (puissance, tool, vitesse, position, etc).
- Système de mesure : *LabView* permet par exemple de réaliser des applications de mesure, et de mettre à disposition les résultats et analyse à disposition d'un autre système.

D'une manière plus générale, les services les plus courants sont :

- le partage de fichiers
- l'accès aux informations du *World Wide Web*
- le courrier électronique
- le partage d'imprimantes
- le commerce électronique
- le stockage en base de données
- le jeu et la mise à disposition de logiciels applicatifs (optique software as a service).

Les serveurs sont utilisés par les entreprises, les institutions et les opérateurs de télécommunication. Ils sont courants dans les centres de traitement de données et le réseau Internet. D'après le cabinet Netcraft, il y a en mars 2014 plus de 376 millions de serveurs web dans le monde, et leur nombre est en augmentation constante depuis l'invention du World Wide Web en 1995.

9.1.4 Client

Dans un réseau informatique, un client est le logiciel qui envoie des demandes à un serveur. Est appelé client aussi bien l'ordinateur depuis lequel les demandes sont envoyées que le logiciel qui contient les instructions relatives à la formulation des demandes et la personne qui opère les demandes.

L'ordinateur client est généralement un ordinateur personnel ordinaire, équipés de logiciels relatifs aux différents types de demandes qui vont être envoyées, comme un navigateur web (client pour le World Wide Web).

Dans le cadre de ce cours, le logiciel *Synaxis* utilisé pour la programmation de l'application robotique est à la fois *client* du système robotique (dans certains cas, plusieurs robots), et des caméras utilisées. Il permet également de fonctionner en tant que serveur afin de mettre à disposition ses *services* à d'autres périphériques/applications. Une même machine peut donc être à la fois *client* et *serveur*.

9.1.5 Adresse physique (MAC)

Pour se connecter au réseau local (Ethernet ou Wi-Fi), un système informatique a besoin d'une *carte réseau*. Cette carte comporte une adresse physique appelée *Mac Adresse* (Media Access Control), qui est unique à travers le monde. Tous les constructeurs de ce type de matériel se répartissent entre eux les adresses disponibles. Ainsi, chaque périphérique *Ethernet*, *Wi-Fi*, *Bluetooth*, ..., possèdent une adresse unique, permettant d'identifier à coup sûr le matériel sur le réseau auquel il est connecté. Cette adresse est utilisée notamment pour l'obtention de l'adresse IP lors de la connexion et se présente sous la forme *01-23-45-67-89-AF*.

Cette adresse est parfois utilisée par les systèmes de protection (licences), et permet d'associer des données à une machine spécifique. Toutefois, si l'adresse est obtenue via le système d'exploitation, sa valeur peut toujours être modifiée par un logiciel mal intentionné. Si elle est obtenue en ligne directe via les drivers de la carte réseau, cette information devient plus robuste.

9.1.6 Adresse IP

L'adresse IP permet d'identifier une machine sur un réseau. Elle doit impérativement être unique sur le réseau considéré. Elle peut être déterminée de 2 manières :

- Manuellement: la personne en charge du réseau détermine l'adresse de chaque machine.
- Automatiquement : lors de son initialisation, la carte réseau demande au serveur *DHCP* (Dynamic Host Configuration Protocol) une adresse IP, qui lui *loue* en retour une adresse unique sur le réseau qu'il administre, pour une durée déterminée. En plus de l'adresse IP, ce serveur DHCP indique au client le serveur des noms, la passerelle par défaut, le nom du réseau, etc. Le réseau de la HE-ARC, comme la plus part des réseaux d'entreprise fonctionnent selon le *DHCP*.

Au laboratoire de robotique, les robots possèdent une adresse IP fixe, ne supportant pas le *DHCP*. Ces machines sont pourtant sur le même réseau que les PC de l'entreprise, et de ce fait, le serveur DHCP du réseau doit connaître ces adresses réservées dans sa plage d'allocation et ne pas les réattribuer à un autre client.

Les camera *Cognex* sont elles configurées en *DHCP*, si bien que lorsqu'elles sont reconnectées au réseau (par exemple après une interruption de l'alimentation), elles peuvent se voir attribuer une nouvelle adresse différente par le serveur *DHCP*. Cela pose un problème, car elles ont la fonction de serveur, et il est alors nécessaire de reconfigurer cette connexion au niveau du client (*Synaxis* par

exemple). Généralement, une machine *serveur* ne devrait pas changer d'adresse pour ces raisons évidentes.

Remarques :

- Le nombre d'adresses IPv4 est de $2^{32} = 4'294'967'296$. A l'échelon mondial, ce nombre atteint ses limites, raison pour laquelle IPv6 a été introduit en 1998 (2^{128} !).
- Les machines de 2 sous-réseaux séparés peuvent avoir les mêmes plages d'adresses, dans la mesure où il n'y a pas de connexion directe vers l'extérieur (*Internet*).
- La *passerelle par défaut* est l'adresse de la machine réalisant la connexion du réseau local vers l'extérieur (*Internet* par exemple) fait également partie des réglages relatifs à l'adresse IP.
- Le *masque de sous-réseau* (subnet mask) permet de définir le *sous-réseau* auquel appartient la machine, ainsi que la plage d'adresse des hôtes. Pour le réseau de la HE-ARC, un PC basé au Locle se voit attribuer :
 - masque de sous-réseau IPv4: 255.255.252.000 (xFFFFFFC00)
 - Le sous-réseau est identifié par le masquage des 22 premiers bits.
 - Le nombre d'hôte possible pour ce sous-réseau est de $x0400 = 1024$.
 - une adresse IPv4 : 157.26.95.28 (x9D1A5F1C)
 - Le sous-réseau est 157.26.92.00 (x9D1A5C00)
 - Le numéro de l'hôte dans ce sous-réseau est 796 (x31C).
- **192.168.0.0** : généralement, un réseau privé (maison, machine) est défini par défaut sur la base de cette adresse. Le routeur prend l'adresse 192.168.0.1, et les machines qui s'y connectent les adresses 192.168.0.2 à 192.168.0.255 (254 adresses possibles). C'est le cas des routeurs domestiques par exemple.
- **127.0.0.1 (localhost)** : cette adresse locale permet à une application *client* de se connecter sur un *serveur* hébergé par la même machine (exécution de tests, etc).

9.1.7 Port

La notion de port logiciel permet, sur un ordinateur donné, de distinguer différents interlocuteurs. Ces interlocuteurs sont des programmes informatiques (*serveur* et *clients*) qui, selon les cas, écoutent ou émettent des informations sur ces ports. Un port est distingué par son numéro ($2^{16} = 65'535$ ports différents).

Cela permet à une même machine (via une même *adresse IP*) d'héberger plusieurs applications *serveur* et/ou *client*. Par convention, les ports 1 à 1023 sont *réservés* aux services de bases, et régulièrement utilisés :

- 20/21 : échange de fichier via FTP
- 23 : telnet
- 25 : SMTP (envoi d'emails)
- 80 : http (navigation sur le web)
- 443 : https

Le numéro de port doit impérativement être spécifié côté serveur et client pour l'établissement d'une connexion TCP/IP. Dans certain cas, il est imposé (*Cognex Insight: telnet*).

9.1.8 Switch Ethernet

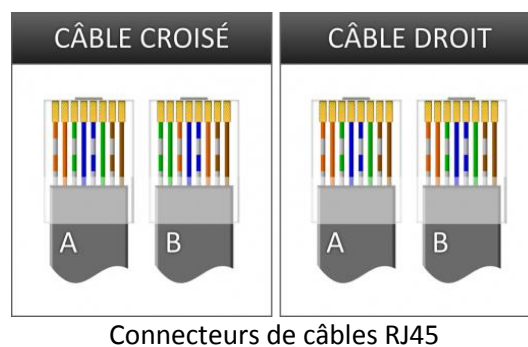
Un commutateur réseau, ou *switch*, est un équipement qui relie plusieurs segments (câbles ou fibres) dans un réseau informatique et de télécommunication et qui permet de créer des circuits virtuels. La commutation est un des deux modes de transport de trame au sein des réseaux informatiques et de communication, l'autre étant le routage. Dans les réseaux locaux (LAN), il s'agit le plus souvent d'un boîtier disposant de plusieurs ports Ethernet (entre 4 et plusieurs centaines), il a donc la même apparence qu'un concentrateur (hub) mais peut être configuré pour un accès direct à internet, ce qui n'est pas possible pour un hub.

9.1.9 Hub Ethernet

Un hub Ethernet ou concentrateur Ethernet est un appareil informatique permettant de concentrer les transmissions Ethernet de plusieurs équipements sur un même support dans un réseau informatique local. Ce type d'appareil a tendance à tomber en désuétude au profit du commutateur réseau.

9.1.10 Câble croisé

Un *câble croisé* permet de relier directement 2 appareils entre eux, sans passer par un *switch* ou un *hub*. Cela est par exemple utile lors d'intervention de dépannage sur un appareil.



Connecteurs de câbles RJ45

9.1.11 DNS, DynDNS

Le Domain Name System (ou DNS, système de noms de domaine) est un service permettant de traduire un *nom de domaine* (www.he-arc.ch) en informations de plusieurs types qui y sont associées, notamment en *adresses IP* de la machine portant ce nom. Ce fonctionnement implique que les serveurs possèdent une adresse IP statique (le plus répandu).

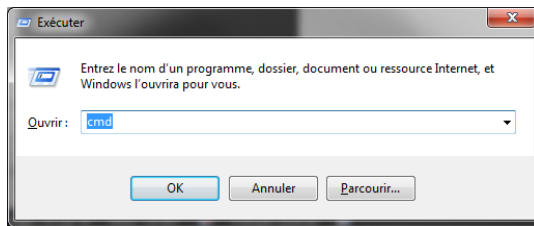
Il existe toutefois une alternative pour associer un nom de domaine à une adresse IP dynamique, mais c'est généralement réservé à des services non-professionnels : *DynDNS* est un service permettant à des utilisateurs qui utilisent une adresse IP dynamique de disposer quand même d'un nom de domaine. Le serveur DNS (Domain Name System) reçoit la nouvelle adresse IP via un petit programme et met à jour le nom de domaine ou de sous-domaine attaché à l'IP.

Il est souvent utilisé par des particuliers qui hébergent leur site web sur leur propre machine mais qui, ne disposant pas d'adresse IP fixe, doivent actualiser régulièrement leur DNS. Un logiciel installé sur cette machine teste l'adresse IP à intervalles réguliers et informe le serveur de DynDNS en cas de changement. Celui-ci met alors à jour les serveurs de DNS.

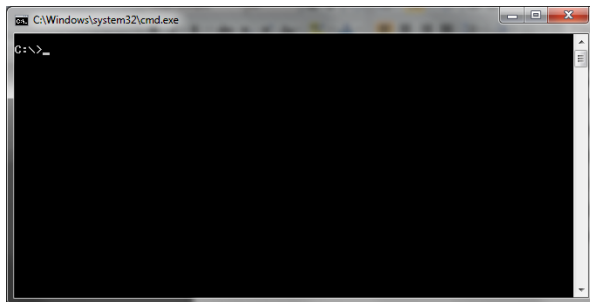
La connexion sur la machine à IP dynamique s'effectue alors par l'intermédiaire du nom de domaine ou de sous-domaine.

9.1.12 Visualiser et manipuler les paramètres réseau sous Windows

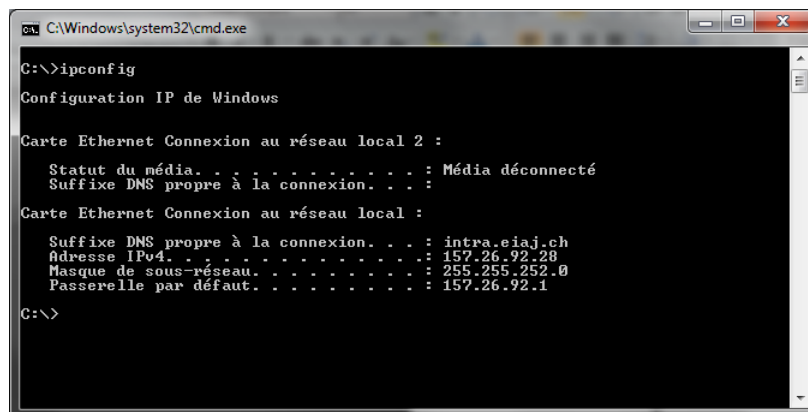
- 1) Ouvrir la fenêtre de commande : écrire "exécuter" dans la recherche du menu, ou touches  + R



- 2) Entrer "cmd" pour ouvrir l'interpréteur de commande *MS DOS*.



- 3) **ipconfig** : affiche les éléments principaux de la configuration IP de Windows, pour chaque *interface réseau* (carte Ethernet, wifi, etc):
 - a. Etat de la connexion (active ou non)
 - b. Adresse IPv4 (ou IPv6).
 - c. Masque de sous-réseau.
 - d. Passerelle par défaut.



- 4) **ipconfig/all** : indique tous les paramètres de la configuration IP de Windows, pour chaque *interface réseau* (carte Ethernet, wifi, etc), notamment :
- Nom de l'hôte → le nom de la machine.
 - L'adresse physique des cartes réseau (MAC adresse).
 - Le mode DHCP

```

C:\Windows\system32\cmd.exe
C:\>ipconfig/all

Configuration IP de Windows

Nom de l'hôte . . . . . : LO-EIN-007-JMA
Suffixe DNS principal . . . . . : intra.eiaj.ch
Type de noeud . . . . . : hybride
Routage IP activé . . . . . : Non
Proxy WINS activé . . . . . : Non
Liste de recherche du suffixe DNS : intra.eiaj.ch

Carte Ethernet Connexion au réseau local 2 :

Statut du média . . . . . : Média déconnecté
Suffixe DNS propre à la connexion . . : 
Description . . . . . : TeamViewer UPN Adapter
Adresse physique . . . . . : 00-FF-73-BA-AD-43
DHCP activé . . . . . : Oui
Configuration automatique activée . . : Oui

Carte Ethernet Connexion au réseau local :

Suffixe DNS propre à la connexion . . : intra.eiaj.ch
Description . . . . . : Intel(R) 82567LM-3 Gigabit Network C
onnection
Adresse physique . . . . . : 00-23-AE-56-15-9E
DHCP activé . . . . . : Oui
Configuration automatique activée . . : Oui
Adresse IPv4 . . . . . : 157.26.92.28<préfééré>
Masque de sous-réseau . . . . . : 255.255.252.0
Bail obtenu . . . . . : vendredi 4 avril 2014 11:22:27
Bail expirant . . . . . : samedi 12 avril 2014 11:22:27
Passerelle par défaut . . . . . : 157.26.92.1
Serveur DHCP . . . . . : 157.26.80.14
Serveurs DNS . . . . . : 157.26.80.30
                          157.26.115.12
Serveur WINS principal . . . . . : 157.26.115.12
Serveur WINS secondaire . . . . . : 157.26.80.30
NetBIOS sur Tcpip . . . . . : Activé

C:\>
  
```

- 5) **arp -a** : affiche la table des différents serveurs auxquels est connectée la machine.
- 6) **ping** : permet de tester la connexion entre la machine et un serveur, en spécifiant son *nom* ou son *adresse* (IPvX). Cette commande envoie plusieurs requêtes vers le serveur, et indique le temps nécessaire pour la réception de la réponse. En cas de problème, le délai d'attente est dépassé, et la perte est de 100%. Cette simple instruction permet ainsi de tester si la connexion entre un serveur et un client est possible d'un point de vue TCP/IP : concordance du masque de sous-réseau, câblage, etc.

```

C:\Windows\system32\cmd.exe
C:\>ping lo-ein-senth
La requête Ping n'a pas pu trouver l'hôte lo-ein-senth. Vérifiez le nom et essayez à nouveau.

C:\>
C:\>ping lo-ein-nimbus

Envoi d'une requête 'ping' sur lo-ein-nimbus.intra.eiaj.ch [157.26.80.73] avec 32 octets de données :
Réponse de 157.26.80.73 : octets=32 temps=1 ms TTL=124
Réponse de 157.26.80.73 : octets=32 temps=1 ms TTL=124
Réponse de 157.26.80.73 : octets=32 temps=1 ms TTL=124
Réponse de 157.26.80.73 : octets=32 temps=1 ms TTL=124

Statistiques Ping pour 157.26.80.73:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 1ms, Maximum = 1ms, Moyenne = 1ms

C:\>ping 157.26.92.1

Envoi d'une requête 'Ping' 157.26.92.1 avec 32 octets de données :
Réponse de 157.26.92.1 : octets=32 temps<1ms TTL=255
Réponse de 157.26.92.1 : octets=32 temps<1ms TTL=255
Réponse de 157.26.92.1 : octets=32 temps<1ms TTL=255
Réponse de 157.26.92.1 : octets=32 temps<1ms TTL=255

Statistiques Ping pour 157.26.92.1:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 0ms, Maximum = 0ms, Moyenne = 0ms

C:\>ping 1.2.3.4

Envoi d'une requête 'Ping' 1.2.3.4 avec 32 octets de données :
Délai d'attente de la demande dépassé.
Délai d'attente de la demande dépassé.
Délai d'attente de la demande dépassé.
Délai d'attente de la demande dépassé.

Statistiques Ping pour 1.2.3.4:
    Paquets : envoyés = 4, reçus = 0, perdus = 4 (perte 100%),

C:\>
  
```

- 7) **tracert (traceroute)** : permet de suivre les chemins qu'un paquet de données va prendre pour aller de la machine locale à une autre machine connectée au réseau IP.

```

C:\Windows\system32\cmd.exe

C:\>tracert www.google.ch

Détermination de l'itinéraire vers www.google.ch [173.194.34.183]
avec un maximum de 30 sauts :

 1    3 ms    <1 ms    <1 ms    gw-lo-ein-1.nms.he-arc.ch [157.26.92.11]
 2    <1 ms    <1 ms    <1 ms    pe-cf-net-1.nms.he-arc.ch [157.26.68.251]
 3    1 ms     2 ms     3 ms    pe-ne-ca2-1.nms.he-arc.ch [157.26.68.941]
 4    1 ms     1 ms     9 ms    gw-ne-ca2-1.nms.he-arc.ch [157.26.68.1251]
 5    3 ms     2 ms     2 ms    accessca2-gi-1-0-5.he-arc.ch [192.135.150.21]
 6    1 ms     1 ms     1 ms    accessa-gi-3-11.unine.ch [192.42.42.1051]
 7    2 ms     1 ms     1 ms    swine2-10ge-3-2.switch.ch [192.42.42.331]
 8    3 ms     3 ms     3 ms    swizh2-10ge-4-3.switch.ch [130.59.36.981]
 9    11 ms    63 ms    29 ms    swilX1-10GE-3-3.switch.ch [130.59.36.1291]
10    16 ms    3 ms     3 ms    equinix-zurich.net.google.com [194.42.48.581]
11    9 ms     15 ms    12 ms    66.249.94.52
12    9 ms     9 ms     9 ms    209.85.251.248
13    17 ms    17 ms    16 ms    209.85.246.40
14    26 ms    31 ms    22 ms    209.85.246.155
15    22 ms    22 ms    45 ms    209.85.253.197
16    22 ms    22 ms    22 ms    209.85.253.175
17    22 ms    21 ms    21 ms    lhr14s22-in-f23.1e100.net [173.194.34.183]

Itinéraire déterminé.

C:\>tracert lo-ein-nimbus

Détermination de l'itinéraire vers lo-ein-nimbus.intra.eiaj.ch [157.26.80.731]
avec un maximum de 30 sauts :

 1    4 ms     4 ms    <1 ms    gw-lo-ein-1.nms.he-arc.ch [157.26.92.11]
 2    4 ms    <1 ms    12 ms    pe-cf-net-1.nms.he-arc.ch [157.26.68.251]
 3    1 ms     1 ms     1 ms    pe-ne-ca2-1.nms.he-arc.ch [157.26.68.941]
 4    2 ms     4 ms     4 ms    gw-ne-ca2-1.nms.he-arc.ch [157.26.68.1251]
 5    2 ms     1 ms     1 ms    lo-ein-nimbus.intra.eiaj.ch [157.26.80.731]

Itinéraire déterminé.

C:\>tracert 157.26.92.1

Détermination de l'itinéraire vers gw-lo-ein-1.nms.he-arc.ch [157.26.92.11]
avec un maximum de 30 sauts :

 1    <1 ms    <1 ms    <1 ms    gw-lo-ein-1.nms.he-arc.ch [157.26.92.11]

Itinéraire déterminé.

C:\>_

```

10 Des connaissances utiles à l'ingénieur

10.1 Loi des similitudes, ou "effet d'échelle"

La loi des similitudes s'exprime partout dans la nature et le monde qui nous entoure. C'est une notion dont tout ingénieur devrait connaître l'existence.

"L'effet d'échelle traite des conséquences physiques venant de la modification de la dimension d'un corps ou plus généralement d'une grandeur physique."

Si les dimensions d'un corps varient en L , la surface varie en L^2 , le volume en L^3 .

Ainsi, quand la dimension d'un corps est doublée :

- Sa surface est multipliée par $2^2 = 4$
- Son volume est multiplié par $2^3 = 8$
- Son poids également par 8 également (matériaux inchangés)
- L'inertie est multipliée par $2^5 = 32$!

Quelques exemples où cette loi des similitudes se met en évidence :

- Monde vivant
 - Soit 2 mammifères, l'un 100x plus grand que l'autre (par exemple un éléphant et un petit rongeur). Le gros animal a un rapport *surface de peau/masse* 100x plus faible ($10^6/10^4$). La dissipation de chaleur est donc beaucoup plus importante pour le petit mammifère, et devra dépenser proportionnellement 100x plus d'énergie pour maintenir sa température, et devrait manger 100x plus d'aliment par rapport à son propre poids. Dans la réalité, un grand mammifère mange 2-7% de l'équivalent de son poids, alors qu'un petit peut manger jusqu'à 100%. Les grandes oreilles de l'éléphant jouent un rôle très important dans la régulation de sa température corporelle, en lien avec son très faible rapport *surface/masse*.
 - Un éléphant ne peut pas sauter : ses os se briseraient au décollage ou à l'atterrissage, en raison d'une résistance (surface des os) trop faible par rapport à sa masse.
 - La fréquence de battement d'aile est nettement plus élevée pour un colibri qu'un aigle.
 - Pour la même raison, les supers héros ne peuvent pas exister : impossible de sauter haut, courir vite, soulever des charges élevées, car la résistance des membres et leur puissance est simplement irréalisable avec les matériaux et technologies connus à ce jour. L'incroyable *Hulk* s'enfoncerait dans le sol, les robots *Transformers* auraient des vitesses de déplacement plus lentes que celle d'un escargot, etc.

- Génie civil
 - Un immeuble a une surface d'échange thermique par appartement plus faible qu'une maison individuelle → économies de chauffage
 - Un câble en acier très long ne supporterait pas son propre poids, tout comme un barrage ou une tour très élevés.
- Physique, technique
 - Ondes sonores : si la taille d'un élément vibrant est modifiée, sa fréquence de résonnance change : impossible de miniaturiser une cloche sans changer son timbre.
 - Une voiture modèle réduit qui tombe d'une table ne sera pratiquement pas endommagée. Dans les mêmes proportions, une voiture réelle tombant d'une falaise sera complètement détruite: la vitesse d'impact est différente, ainsi que l'énergie et la résistance du véhicule par rapport à sa masse.
 - Un réacteur d'avion miniaturisé, sans changement dans sa conception ne fonctionne pas, de même qu'un moteur de montre à grande échelle !
- Robotique
 - Un robot de petite taille (rayon de l'*enveloppe de travail* < 0.4 m) a généralement une très grande dynamique (*vitesse maximum de déplacement* de l'axe élevée), et une *charge transportable admissible* faible (~ 0.5 kg). La masse du robot est de quelques dizaines de kg.
 - Un robot de grande taille (> 3 m) a une vitesse de déplacement faible, et une capacité de charge très grande (500-1000 kg). Sa masse est de plusieurs tonnes.

Remarque : dans le cas d'un robot dont les articulations sont de *type angulaire*, La vitesse linéaire du préhenseur est fonction du rayon et de la vitesse angulaire de l'axe. Elle peut donc être relativement élevée dans le cas d'un robot de grande taille, même si l'axe se meut lentement. L'inertie observant une variation en L^5 , les moteurs et réducteurs doivent être beaucoup plus puissant pour conserver un pouvoir d'accélération respectable.

Sources

Wikipedia

http://fr.wikipedia.org/wiki/Programmation_proc%C3%A9durale

http://en.wikipedia.org/wiki/Stack_%28abstract_data_type%29

http://fr.wikipedia.org/wiki/Langage_de_programmation#Paradigmes

<http://fr.wikipedia.org/wiki/Sous-r%C3%A9seau>

<http://fr.wikipedia.org/wiki/Ethernet>

<http://fr.wikipedia.org/wiki/Internet>

http://fr.wikipedia.org/wiki/R%C3%A9seau_local

http://fr.wikipedia.org/wiki/Port_%28logiciel%29

http://fr.wikipedia.org/wiki/Commutateur_r%C3%A9seau

http://fr.wikipedia.org/wiki/Domain_Name_System